МЕТОДИЧКА · ПРАКТИЧЕСКИЙ КУРС · 2026

Rust для C++ и Go-разработчика от нуля до embedded

Полная программа курса: 76 занятий по 15–30 минут с теорией, практикой и домашними заданиями, пять лабораторных работ и семестровый проект. От владения и заимствования — через алгоритмы и CRUD-сервисы на Axum — к прошивкам no_std для Raspberry Pi и ESP32.

16	76	5	1
НЕДЕЛЬ	ЗАНЯТИЙ	ЛАБОРАТОРНЫХ	ИТОГОВЫЙ ПРОЕКТ

м1–4 **фундамент:** владение, borrow checker, типы, ошибки, трейты

м3 **алгоритмы:** сортировки, графы, ДП — на родных структурах Rust

м4–5 **сервисы:** async/Tokio, CRUD на Axum + SQLx + PostgreSQL

м6–7 **железо:** GPIO/UART на Raspberry Pi, no_std и Embassy на ESP32

м8 **проект:** сквозной IoT-сценарий «датчик → API → база»

КУРС ДЛЯ ПРАКТИКУЮЩИХ РАЗРАБОТЧИКОВ

16 НЕДЕЛЬ · 2026

ВВЕДЕНИЕ

Как устроена эта методичка

Для кого этот курс. Вы — практикующий разработчик на C++ или Go, который хочет добавить Rust в рабочий арсенал и довести знание языка до трёх прикладных вершин: уверенно писать основные алгоритмы и структуры данных, собирать полноценные CRUD-приложения с базой данных и разрабатывать софт для встраиваемых платформ уровня Raspberry Pi и ESP32. Методичка не объясняет, что такое переменная или HTTP-запрос, с нуля — она постоянно опирается на ваш опыт: каждая новая концепция Rust сопоставляется с тем, чем вы уже пользуетесь — RAII и `unique_ptr` из C++, горутинами и интерфейсами из Go.

Формат занятия. Каждое занятие рассчитано на 15–30 минут видео или живой лекции и всегда состоит из четырёх частей: короткая теория с рабочим примером кода, разбор типичной ошибки компилятора, практическое упражнение под руководством и домашнее задание. Домашняя работа занимает 30–60 минут и почти всегда превращает пример из занятия в маленький законченный артефакт: утилиту, модуль библиотеки или тест. Итоговая нагрузка — около 4–6 часов в неделю, из которых половина — практика за клавиатурой.

Три правила прогресса. Первое: всё, что встречается в теории, набирается руками — Rust не читается, Rust пишется. Второе: каждая лабораторная сдаётся через код-ревью по чек-листу модуля, поэтому репозиторий с историей коммитов ведётся с первой недели. Третье: ошибка компилятора — это не препятствие, а материал; на самых частых ошибках (E0382, E0502, E0277, E0106) построены отдельные упражнения, и к концу модуля 1 вы должны читать сообщения rustc быстрее, чем гуглить их.

Что нужно подготовить заранее. Рабочая машина с Linux, macOS или WSL2; установленный `rustup` (стабильный канал), VS Code с расширением rust-analyzer; для модуля 5 — PostgreSQL 15+ и Docker; для модуля 6 — Raspberry Pi 3/4/5 (или аналог: Orange Pi, Rock Pi) с Linux на борту и светодиодно-кнопочный набор; для модуля 7 — отладочная плата ESP32-DevKitC или ESP32-C3-DevKit, кабель USB-UART, датчик на шине I2C (BMP280, BME280 или аналог). Если железа пока нет, оба embedded-модуля можно пройти в симуляторах: QEMU для Pi и `esp-hal`-совместимые эмуляторы для ESP32, — но лабы сдают с реальной платой, поэтому лучше её приобрести к 12-й неделе.

НАВИГАЦИЯ

Карта курса: три трека, восемь модулей

Курс построен линейно, но делится на три смысловых трека. Языковой трек (модули 1–2) отвечает на вопрос «как на этом писать вообще». Инженерный трек (модули 3–5) превращает знание языка в способность писать алгоритмы и сервисы. Системный трек (модули 6–8) выводит код за пределы десктопа — на одноплатники и микроконтроллеры. Пропускать модули не рекомендуется: терминальный проект опирается на все лабы сразу.

Модуль	Название	Недели	Занятий	Результат модуля
M1	Фундамент Rust для C++/Go-разработчика	1–4	28	Читаете ошибки компилятора, владеете памятью без GC и вручную
M2	Система типов в полную силу	5–6	10	Трейты, генерики, итераторы и умные указатели — свойственным Rust способом
M3	Алгоритмы и структуры данных на Rust	7–8	10	Пишете и бенчмарките сортировки, поиск, графы, ДП на идиоматичном Rust
M4	Конкурентность и асинхронность	9	5	Потоки, каналы, Send/Sync, async/await и Tokio без страха
M5	CRUD-приложение на Axum + SQLx	10–12	15	Рабочий REST API с PostgreSQL, JWT, тестами и docker-compose
M6	Rust на Raspberry Pi	13	4	Кросс-компиляция, GPIO, UART/I2C, systemd-демон с датчиком
M7	ESP32 и no_std	14	4	Bare-metal прошивка: blink, Embassy async, I2C-датчик, Wi-Fi/MQTT
M8	Семестровый проект	15–16	2 спринта	Сквозная система «узел → API → БД» + защита

ПЛАНИРОВАНИЕ

Календарный план-график

Таблица ниже — календарь на семестр из расчёта 5 учебных дней в неделю. Столбец «Контроль» фиксирует, что именно сдаётся в конце недели: ДЗ проверяются самопроверкой по чек-листу, лабораторные — код-ревью. Если неделя выпадает (отпуск, релиз), календарь просто сдвигается: материал не сцеплен жёстко с датами.

Неделя	Тема недели	Занятия	Контроль в конце недели
1	Старт: toolchain, синтаксис, типы, match	1.1–1.7	ДЗ: конвертер температур + fizzbuzz на match
2	Владение, заимствование, borrow checker	1.8–1.14	ДЗ: разбор 8 сниппетов «почему не компилируется»
3	Времена жизни, String/&str, коллекции	1.15–1.21	Лаба 1 старт: rsgrep (каркас, чтение файлов)
4	Ошибки, Result, модули, тестирование	1.22–1.28	**Сдача лабы 1** (rsgrep) + ДЗ: рефакторинг
5	Трейты, генерики, трейт-объекты	2.1–2.5	ДЗ: свой трейт + генерик-функция largest
6	Замыкания, итераторы, умные указатели	2.6–2.10	ДЗ: пайплайн обработки логов на итераторах

7	Сложность, сортировки, бинарный поиск	3.1–3.5	Лаба 2 старт: сортировки с бенчмарками
8	Деревья, кучи, графы, ДП	3.6–3.10	**Сдача лабы 2** (алгоритмическая библиотека)
9	Потоки, каналы, async/await, Tokio	4.1–4.5	ДЗ: воркер-пул на каналах
10	Обзор web-фреймворков: Actix, Rocket, Axum	5.1–5.5	ДЗ: один мини-API на двух фреймворках
11	SQLx: схема БД, запросы, транзакции	5.6–5.10	Лаба 3 старт: репозиторий-слой
12	Полный REST API: валидация, JWT, тесты	5.11–5.15	**Сдача лабы 3** (CRUD API)
13	Raspberry Pi: кросс-компиляция, GPIO, I2C	6.1–6.4	**Сдача лабы 4** (Pi-метеостанция)
14	ESP32: no_std, esp-hal, Embassy, Wi-Fi	7.1–7.4	**Сдача лабы 5** (ESP32-узел)
15	Проект, спринт 1: архитектура и MVP	спринт	Репо проекта, работающий MVP, демо-заметка
16	Проект, спринт 2: полировка и защита	спринт	**Защита проекта** (демо 10 мин + код-ревью)

АТТЕСТАЦИЯ

Критерии оценки и допуск к проекту

Итоговая оценка складывается из четырёх составляющих. Домашние задания сдаются еженедельно и проверяются по принципу «сделано / не сделано» с обязательным самопрогоном чек-листа модуля. Лабораторные работы принимаются только через код-ревью: рецензент (преподаватель или напарник по курсу) проверяет идиоматичность, тесты и историю коммитов. Семестровый проект оценивается комиссией или наставником по таблице ниже.

Компонент	Вес	Что проверяется
Домашние задания (12 недель)	20%	Выполнение всех ДЗ; допускается 2 «долга», закрываемых в течение следующей недели
Лабы 1–5	40%	По 8% за лабу: компиляция без warnings, тесты, идиоматичность, чистота истории коммитов
Семестровый проект	35%	Работоспособность (10), архитектура и идиоматичность (10), тесты и CI (5), документация и защита (10)
Чек-листы самопроверки	5%	Заполненные чек-листы всех 8 модулей (форма — в конце каждого модуля)

Порог допуска к проекту: сданы лабы 1–3 минимум на 60 баллов из 100 каждая, лабы 4–5 зачтены хотя бы на «зачёт» (работает на реальном железе), закрыто не более одного долга по ДЗ. Студент, не прошедший порог к 15-й неделе, выполняет упрощённый проект-минимум: CRUD API без embedded-части, но с полным набором тестов.

ПРАВИЛО ЧЕСТНОСТИ

Копировать код лаб разрешено только с указанием источника в комментарии и только на уровень «фрагмент, а не решение». За целиком скопированную лабу без понимания защита проекта всё равно провалится — проекты защищаются лично, с вопросами «почему здесь Arc».

МОДУЛЬ 1 · НЕДЕЛИ 1–4 · ЗАНЯТИЯ 1.1–1.28

Фундамент Rust для C++/Go-разработчика

4 недели | 28 занятий | ≈ 18 ч теории | лаба 1: rsgrep | порог всего курса

Первый модуль — самый важный и самый плотный. Здесь закладывается то, что отличает Rust и от C++, и от Go: модель владения вместо ручного управления памятью и вместо сборщика мусора, исчерпывающий `match` вместо switch, система ошибок на `Result` вместо исключений и `err`-возвратов. Материал четырёх недель выстроен так, что каждая новая тема опирается на предыдущую: типы — на синтаксис, владение — на типы, времена жизни — на заимствование, ошибки — на всё сразу. Торопиться нельзя: студент, «проехавший» недели 2–3 по инерции, упирается в borrow checker в модуле 5 и откатывается назад уже с большими потерями времени.

НЕДЕЛЯ 1 Старт: toolchain, синтаксис, типы, match

УРОК 1.1 20 мин

Зачем Rust, если есть C++ и Go

ЦЕЛЬ — увидеть место Rust в нише «производительность C++ + безопасность Go» и понять, ради чего платить стоимостью освоения borrow checker.

Rust занимает нишу системных языков, где до него доминировал C++: нулевая стоимость абстракций, ручной контроль памяти, отсутствие рантайма по умолчанию. Но, в отличие от C++, безопасность памяти проверяется компилятором на этапе сборки: use-after-free, двойное освобождение и висячие указатели в безопасном Rust невозможны в принципе — они не проходят компиляцию. От Go Rust наследует современный инструментарий (cargo, единый форматтер, тесты из коробки), выражения-итераторы и дешёвую конкурентность, но не платит за это GC-паузами и рантаймом в мегабайты — потому Rust работает там, где Go не ставится: на микроконтроллерах, в ядре Linux, в драйверах файловой системы.

ЧТО КОМПИЛЯТОР ЗАПРЕЩАЕТ ЕЩЁ ДО ЗАПУСКА

```
fn main() {
    let data = vec![1, 2, 3];
    let stolen = data;            // владение переехало в stolen
    println!("{}", data[0]);      // ошибка E0382: borrow of moved value
}                                 // в C++ это сегфолт, в Go — незаметная аллокация
```

СРАВНЕНИЕ

C++ даст то же управление памятью, но через дисциплину RAII и умные указатели, которую компилятор не проверяет. Go перекладывает ответственность на GC: удобно для сервисов, дорого для embedded и низколатентных систем. Rust кодирует правила прямо в систему типов.

ПРАКТИКА Пробежаться по https://rust-lang.org и blessed.crates.rs; выписать 5 проектов, которые вы уже используете и которые написаны на Rust (подсказка: ripgrep, fd, deno, polkadot, rustc).

ДОМАШКА Написать короткий меморандум (10–15 строк): какие из ваших текущих задач на C++/Go выиграли бы от Rust, а какие — нет, и почему.

УРОК 1.2 25 мин

Toolchain: rustup, cargo, rust-analyzer

ЦЕЛЬ — собрать рабочее окружение, понять роли каждого инструмента и структуру проекта.

`rustup` — менеджер версий компилятора: он ставит стабильный, бета и ночной каналы и целевые платформы (targets), которые понадобятся в модуле 6 для кросс-компиляции на Raspberry Pi. `cargo` — система сборки и менеджер пакетов одновременно: как make + vcpkg/conan в мире C++ или go build + go mod в мире Go, но единым бинарём. `rust-analyzer` — language-сервер, дающий IDE-функции в VS Code; без него работа с Rust в 2026 году не считается настроенной. Проект — это crate: папка с `Cargo.toml` и исходниками в `src/`.

ТЕРМИНАЛ · УСТАНОВКА И ПЕРВЫЙ ПРОЕКТ

```
rustup default stable          # канал по умолчанию
cargo new hello_rust           # новый бинарь: src/main.rs + Cargo.toml
cd hello_rust && cargo run     # сборка + запуск
cargo check                    # проверка типов без генерации бинар (быстро!)
cargo clippy                   # линтер: 700+ правил идиоматичности
```

> СРАВНЕНИЕ
> Go-разработчику: `cargo` = `go build/get/test/mod` в одной команде. C++-разработчику: cargo убирает cmake-зоопарк — зависимости скачиваются и собираются сами, версия фиксируется в Cargo.lock.

ДОМАШКА Установить toolchain, создать проект, добавить зависимость `rand`, написать программу, выводящую случайное число, и прогнать `cargo fmt` + `cargo clippy` без warnings.

УРОК 1.3 20 мин

Cargo.toml, зависимости, features, workspace

ЦЕЛЬ — научиться читать манифест проекта и управлять зависимостями осознанно.

`Cargo.toml` описывает пакет: имя, версию, edition (языковая версия: 2021 и 2024 отличаются правилами заимствования замыканий и лексикой), зависимости с semver-ограничениями. Секция `[features]` — это условная компиляция уровня зависимостей: библиотека может включать дополнительные модули (например, поддержку TLS) по флагу. Workspace — группа крейтов в одном репозитории с общим Cargo.lock, аналог Go-workspace или суперпроекта CMake.

CARGO.TOML

```
[package]
name = "rsgrep"          # лаба 1 начнётся уже в 3-й неделе
version = "0.1.0"
edition = "2021"
```

```
[dependencies]
anyhow  = "1"            # удобные цепочки ошибок (модуль 4)
regex   = "1"            # semver: любая 1.x, точная версия — в Cargo.lock
clap    = { version = "4", features = ["derive"] }  # CLI с деривацией
```

ЧАСТАЯ ОШИБКА НОВИЧКА

Писать версию без кавычек-ограничителей вида `regex = "1.5"` и ожидать точной версии: cargo берёт наибольшую совместимую 1.x. Точная версия живёт в Cargo.lock — его коммитят для бинарей и не коммитят для библиотек.

ДОМАШКА Изучить Cargo.toml и Cargo.lock трёх популярных крейтов из crates.io (например, serde, tokio, axum): какие у них зависимости, какие features существуют и зачем.

УРОК 1.4 25 мин

Синтаксис-скороговорка: let, mut, функции, выражения

ЦЕЛЬ — пройти базовый синтаксис быстро, но с единственным нетривиальным акцентом: «всё есть выражение».

Переменные объявляются через `let` и по умолчанию неизменяемы — мутабельность запрашивается явно словом `mut`. Это переворачивает привычку из C и Go: компилятор заставляет думать, какие данные действительно меняются. Типы выводятся, но указываются при объявлении функций. Главное отличие синтаксиса: тело функции — это выражение, последний блок без точки с запятой и есть возвращаемое значение; `return` нужен только для досрочного выхода.

```
fn bmi(weight_kg: f64, height_m: f64) -> f64 {
    weight_kg / (height_m * height_m)   // без ';' — это выражение-возврат
}

fn classify(w: f64, h: f64) -> &'static str {
    let index = bmi(w, h);
    if index < 18.5 { "недостаток" }        // if — тоже выражение
    else if index < 25.0 { "норма" }
    else { "избыток" }                       // без else не скомпилируется в let
}

fn main() {
    let x = 5;          // неизменяемо навсегда
    let mut y = 5;      // мутабельно
    y += 1;
    println!("{x} {y} {:?}", classify(70.0, 1.75)); // inline-подстановка {x}
}
```

СРАВНЕНИЕ

C++: `const` — опция, Rust: неизменяемость — норма, `mut` — опция. Go: все объявления через `:=` / `var`, мутабельность всегда разрешена — разницы, которую заметит компилятор Rust, в Go нет.

ПРАКТИКА Переписать функцию classify так, чтобы она возвращала не строку, а значение через `let verdict = if ...`, и убедиться, что без ветки else компилятор ругается.

ДОМАШКА Утилита `tempconv`: CLI-программа с тремя подкомандами c2f / f2c / k2c; использовать только std, выводить с точностью до одного знака.

УРОК 1.5 20 мин

Скалярные типы: целые, дробные, bool, char

ЦЕЛЬ — знать фиксированные размеры типов, поведение при переполнении и правила приведения.

Rust задаёт размеры явно: `i8/i16/i32/i64/i128/isize` и беззнаковые собратья; `isize/usize` — размер указателя платформы, как `ptrdiff_t` и `size_t` в C. Литералы читаются с подчёркиваниями и суффиксами: `1_000_000u64`. Переполнение в debug-сборке — паника, в release — заворачивание по модулю; полагаться на это нельзя, для осознанных операций есть методы `checked_add`, `wrapping_add`, `saturating_add`. Неявных приведений целых почти нет — только через `as` (усечение без предупреждения!) или через `try_from` с проверкой.

```
fn main() {
    let small: i8 = 127;
    // let boom = small + 1;            // debug: panic 'attempt to add with overflow'
    println!("{}", small.wrapping_add(1));   // -128: осознанное заворачивание
    println!("{}", small.saturating_add(1)); // 127: насыщение
    let big = 300i32 as u8;             // усечение: 44 — молча!
    let safe: Option<u8> = u8::try_from(300i32).ok(); // None: честная ошибка
    println!("{big} {safe:?}");
}
```

СРАВНЕНИЕ

В C++ арифметика малых типов молча повышается до int, и переполнение signed — UB. В Go переполнение всегда заворачивает. В Rust выбор за вами, но каждый путь назван явно.

ДОМАШКА Написать функцию `avg(values: &[u32]) -> Option<f64>`, которая не переполняется на сумме и возвращает None для пустого среза; проверить на данных с максимумом u32.

УРОК 1.6 25 мин

Кортежи, массивы, структуры и enum

ЦЕЛЬ — освоить составные типы и главное концептуальное отличие: enum как размеченное объединение.

Кортежи — анонимные наборы разнотипных значений с доступом по индексу `t.0`. Массивы `[T; N]` имеют размер, известный на компиляции, и лежат на стеке — в отличие от Go, где срез — это отдельный тип, здесь фикс-размер и динамический вектор строго разделены. Структуры — как в C, но без наследования: только композиция. Настоящая звезда — `enum`: каждое значение — строго один из вариантов, и варианты могут нести данные. Это sum type (tagged union), который в C++ собирался вручную из union + тег, а в Go эмулировался интерфейсами и типовыми свитчами.

```
enum Shape {
    Circle { radius: f64 },          // вариант со структурой
    Rect(f64, f64),                  // вариант с кортежем
    Point,                           // вариант без данных
}
```

```rust
fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle { radius } => 3.14159 * radius * radius,
        Shape::Rect(w, h)        => w * h,
        Shape::Point             => 0.0,
    }   // компилятор требует рассмотреть ВСЕ варианты
}

structSensor { id: u16, value: f32 }    // точный размер, без vtable
```

СРАВНЕНИЕ

Go: `type Shape interface{}` + type switch — проверки в рантайме, можно забыть ветку. C++: `std::variant` + `std::visit` или union вручную с риском чтения не того поля. Rust-enum: исчерпывающая проверка на компиляции.

ДОМАШКА Смоделировать enum-ом состояние термостата: `Idle`, `Heating { power_pct: u8 }`, `Error { code: u16 }`; написать функцию `describe` через match; добавить новый вариант `Defrost` и увидеть ошибку компилятора во всех match.

УРОК 1.7 25 мин

Управление потоком: if, match, loop, while let

ЦЕЛЬ — довести match до автоматизма: шаблоны, связывание, guard-условия.

`match` в Rust — не switch: ветки — это полноценные шаблоны, умеющие разбирать структуры, кортежи, диапазоны и связывать переменные. Исчерпываемость означает, что добавление варианта enum ломает компиляцию во всех матчах без wildcard — это защита, которая экономит часы при рефакторинге. Полезны guard-условия `if` внутри ветки, связывание `@` и опережающие шаблоны. Для простых случаев есть `if let` и `let else` (edition 2021+), для циклов с возвратом значения — `loop { break value }`.

```rust
enum Msg { Temp(u16), Alert { code: u8, critical: bool }, Ping }

fn handle(m: Msg) -> &'static str {
    match m {
        Msg::Temp(t) if t > 850 => "перегрев!",          // guard
        Msg::Temp(_)            => "норма",
        Msg::Alert { code: 0..=9, critical: true } => "критично",
        Msg::Alert { .. }       => "предупреждение",
        Msg::Ping               => "pong",
    }
}

fn poll_until(ok: bool) -> u32 {
    let mut n = 0;
    loop {
        n += 1;
        if n > 3 && ok { break n; }   // loop как выражение
    }
}
```

ЛОВУШКА

Wildcard `_ => {}` в конце гасит пользу исчерпываемости. Правило: wildcard допустим, только когда варианты по смыслу действительно эквивалентны (например, `Msg::Temp(_)` до guard-ветки — это не wildcard, а конкретный шаблон).

ПРАКТИКА FizzBuzz на match с диапазонами и кортежем `(n % 3, n % 5)` — без единого if.

ДОМАШКА Конвертер температур из урока 1.4 перевести на clap с подкомандами и match по аргументам; прогнать clippy до нуля warnings.

НЕДЕЛЯ 2 Владение и заимствование — сердце языка

УРОК 1.8 30 мин

Модель памяти: стек, куча и три философии

ЦЕЛЬ — зафиксировать модель памяти Rust и сопоставить три стратегии управления ресурсами.

Как и в C++, локальные данные в Rust живут на стеке, а динамические — в куче: `Box::new` явно помещает значение в кучу, и это отличие от Go, где компилятор сам решает escape-анализом, куда положить переменную. Освобождение детерминированное: значение уничтожается в конце области видимости, вызывается `Drop` — это RAII из C++, но с проверкой компилятором, что никто не удерживает ссылку на уже убитый объект. Три философии: C++ доверяет программисту (и за это платит UB), Go перекладывает работу на GC (и платит паузами и рантаймом), Rust делает правила владения частью системы типов (и платит кривой обучения).

```
struct Session { user_id: u32 }

impl Drop for Session {
    fn drop(&mut self) {
        println!("сессия {} закрыта", self.user_id);  // детерминированно!
    }
}

fn main() {
    {
        let s = Session { user_id: 42 };
        println!("работаем");
    }                    // ← Drop вызывается ЗДЕСЬ, как в C++ RAII
    println!("после блока");   // уже без сессии, без GC и без утечки
}
```

СРАВНЕНИЕ

C++: `std::unique_ptr<Session>` — та же семантика, но компилятор не следит за сырыми `Session*`, которые вы создали мимо умного указателя. Go: `defer` — вручную для каждого ресурса; память вообще не ваша забота, а вот латентность GC — уже да.

ДОМАШКА Класс `Timer`: в `Drop` печатает, сколько микросекунд жил объект (использовать `std::time::Instant`). Вложить таймеры в блоки и проверить порядок вызова Drop.

УРОК 1.9 30 мин

Владение: move, Copy и Clone

ЦЕЛЬ — понять, когда значение копируется, когда переезжает и почему «движение» обнуляет старую переменную.

Присваивание и передача в функцию по умолчанию **перемещают** значение: старая переменная становится недействительной, и компилятор проследит, чтобы вы не тронули мёртвую. Это move-семантика C++11, но она здесь — единственное поведение по умолчанию, а не опция. Типы простой байтовой структуры (целые, float, bool, char, массивы из

них, кортежи из них) помечены трейтом `Copy` и копируются неявно — как структуры-POD в C++. Для остальных копирование — явное `.clone()`, которое ничем не прикрыто: видно, где дорогая аллокация.

```
#[derive(Clone, Debug)]
struct Config { name: String, retries: u32 }

fn consume(c: Config) { println!("забрал {:?}", c.name); }

fn main() {
    let a = Config { name: "prod".into(), retries: 3 };
    let b = a.clone();          // явная глубокая копия
    consume(a);                 // move: a больше нельзя трогать
    // println!("{a:?}");       // E0382: value borrowed after move
    let mut s = String::from("hello");
    let t = s.clone();          // копия кучи
    s.push_str(" world");       // ок: t — независимая
    let x = 5u32;
    let y = x;                  // Copy: обе живы
    println!("{t} {y} {b:?}");
}
```

> СРАВНЕНИЕ
>
> Go: присваивание структур копирует байты, строки и срезы делят данные — неявная шареность, источник гонок. C++: `auto b = a;` вызывает конструктор копии, а `std::move(a)` оставляет a в «валидном неопределённом» состоянии — Rust делает это состояние типом.

ПРАКТИКА Написать функцию `fn takes(s: String)` и три вызова подряд одной переменной; прочитать ошибку E0382, затем починить тремя способами: clone, передача ссылки, передача значения в цикле.

ДОМАШКА Реализовать `#[derive(Copy, Clone)]` на struct с полем String — объяснить (текстом в комментарии), почему это не компилируется, и какой вывод о Copy следует.

УРОК 1.10 30 мин

Заимствование: & и &mut, правила алиасинга

ЦЕЛЬ — освоить ссылки как способ пользоваться значением без завладения им, и два железных правила.

Ссылка — это указатель с проверкой времени жизни: вы «заимствуете» значение, не забирая владение. Правил два, и они исключают целые классы багов: (1) в любой момент может существовать сколько угодно неизменяемых ссылок `&T` **или** ровно одна изменяемая `&mut T`, но не одновременно; (2) ссылка никогда не живёт дольше владельца. Первое правило на корню убивает data race в однопоточном коде и заставляет архитектуру явно разделять чтение и запись; второе — use-after-free.

```
fn push_all(dst: &mut Vec<i32>, src: &[i32]) {
    for &v in src { dst.push(v); }
}

fn main() {
    let mut v = vec![1, 2, 3];
    let r1 = &v;                // сколько угодно читателей
    let r2 = &v;
    println!("{r1:?} {r2:?}");  // читатели закончились здесь (NLL)
    let w = &mut v;             // теперь один писатель
    w.push(4);
    push_all(&mut v, &[5, 6]);  // повторный уникальный заём
```

```
println!("{v:?} {w:?}");
}
```

СРАВНЕНИЕ

Go: любой может держать указатель и писать когда хочет — гонка ловится только в рантайме (`-race`). C++: `const T&` — соглашение, обходится через `const_cast` одним движением. Rust: правила enforced системой типов.

NLL

С 2018 года действует Non-Lexical Lifetimes: заём заканчивается на последнем использовании, а не на конце фигурных скобок. Поэтому код выше компилируется, хотя r1 и w «живут в одном блоке».

ДОМАШКА Реализовать функции `max_in_place` (получает `&mut Vec<i32>`, заменяет минимальный элемент на максимальный) и `find_all` (возвращает Vec индексов по предикату на `&[i32]`); обе — с юнит-тестами.

УРОК 1.11 25 мин

Borrow checker: разбор типовых отказов

ЦЕЛЬ — научиться читать ошибки E0502/E0506/E0106 и переписывать код, а не бороться с компилятором.

Borrow checker отказывает не из вредности: почти каждый отказ указывает на реальную или потенциальную алиасность. Типовой случай — мутировать коллекцию, пока вы держите ссылку на её элемент: в Go это паника или тихая порча, в Rust — ошибка на компиляции. Лечение почти всегда одно из четырёх: сузить область заёма, скопировать нужные данные (`clone` или индексы вместо ссылок), переиспользовать итераторы или перестроить структуру так, чтобы чтение и запись не пересекались.

```
fn main() {
    let mut names = vec![String::from("ada"), String::from("grace")];
    let first = &names[0];            // заём 1
    names.push(String::from("linus")); // E0502: push требует &mut,
    // println!("{first}");           // а first ещё жив по NLL-правилу
    // Фикс: закончить с first до мутации
    println!("{first}");
    names.push(String::from("linus"));

    // Классика: обновить по ссылке из той же структуры
    let mut cfg = (String::from("host"), 8080u16);
    let host = &cfg.0;
    // cfg.1 = 9090;                  // E0502? Нет: поле другое — ок!
    println!("{host} {}", cfg.1);      // разные поля — независимые заёмы
}
```

КУЛЬТУРНЫЙ МОМЕНТ

Студенты C++ начинают лечить всё `clone()`. Правильная последовательность: сначала сузить заём, потом индексы/дубликаты данных, и только потом clone — как последний инструмент, а не первый.

ДОМАШКА Серия из 8 сниппетов: 4 не компилируются (E0502, E0382, E0506, E0499), 4 компилируются. Для каждого — вердикт «компилируется/нет» и однострочное объяснение. Затем каждый невалидный починить минимальной правкой.

УРОК 1.12 25 мин

Срезы: &[T] и &str

ЦЕЛЬ — освоить срез как «fat pointer» и понять, почему сигнатуры функций пишутся именно через срезы.

Срез `&[T]` — это указатель на данные + длина (два машинных слова, «fat pointer»): вид в куче или на стеке без владения ими. Срез — Rust-аналог Go-среза минус изменяемая длина и минус капасити: подсмотреть можно всё, менять размер — нельзя. Строковый срез `&str` — то же самое для UTF-8-байт. Отсюда золотое правило API: функции принимают `&str` и `&[T]`, а возвращают `String` и `Vec<T>`, если должны владеть результатом. Тогда `String`, `&String`, литералы и `Box<str>` одинаково хорошо проходят в одну функцию через deref-коэрцию.

```
fn count_vowels(s: &str) -> usize {
    s.chars().filter(|c| matches!(c, 'a'|'e'|'i'|'o'|'u')).count()
}

fn main() {
    let owned = String::from("abracadabra");
    let literal = "xyzzy";
    println!("{} {}", count_vowels(&owned), count_vowels(literal));

    let data = [10, 20, 30, 40, 50];
    let mid = &data[1..4];              // срез по диапазону
    println!("{} {:?} {}", mid.len(), mid, std::mem::size_of_val(&mid));
    // size_of_val(&mid) == 16: ptr + len, без капасити
}
```

> СРАВНЕНИЕ
>
> Go: `[]byte` и `string` конвертируются копированием, срез таскает len+cap и может расти. C++: `std::string_view` — ближайший родственник &str, но без гарантии жизни под ним. Rust: гарантия — часть системы типов.

ДОМАШКА Функция `longest_word(text: &str) -> &str`: вернуть самое длинное слово без аллокаций. Тесты: пустая строка, повторяющиеся длины, UTF-8 (слово «ёжик»).

УРОК 1.13 30 мин

Времена жизни: 'a и правила elision

ЦЕЛЬ — понять аннотации lifetime как описание связей, а не «увеличение жизни», и три правила элиминации.

Аннотация `'a` не продлевает ничью жизнь — она лишь описывает компилятору связь: «возвращаемая ссылка живёт не дольше, чем оба входа». В большинстве случаев компилятор выводит это сам по трём правилам elision (каждый ссылочный параметр получает свой lifetime; если один — `&self`, он достаётся возврату; несколько входов — нужен явный `'a`). Явные аннотации нужны в сигнатурах функций с несколькими ссылочными аргументами и в структурах, хранящих ссылки.

```
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() ≥ b.len() { a } else { b }
}

struct Parser<'a> {
    src: &'a str,          // структура заимствует источник
    pos: usize,
}
```

```rust
impl<'a> Parser<'a> {
    fn rest(&self) -> &'astr { &self.src[self.pos..] } // элиминация: 'a от &self... точнее от src
}

fn main() {
    let s1 = String::from("длинная строка");
    let r;
    {
        let s2 = String::from("короткая");
        r = longest(&s1, &s2);
        println!("{r}");       // ок: r использована, пока s2 жив
    }
    // println!("{r}");         // E0597: s2 умерла, r не может жить дольше
}
```

> СРАВНЕНИЕ
>
> C++: `std::string_view` из временного объекта — висячий указатель, молча. Go: ничем не защищён от удержания указателя на элемент, который перепакуют. Rust: связь вычисляется и проверяется до запуска.

ДОМАШКА Реализовать структуру `Splitter<'a>` с методом `next(&mut self) -> Option<&'a str>` — итератор по словам строки-источника без аллокаций. Тесты на пустую строку и несколько пробелов.

УРОК 1.14 20 мин

Лаба 1 · старт: каркас rsgrep

ЦЕЛЬ — заложить каркас лабораторной №1: утилита поиска по строкам в файлах (собственный grep) и закрепить неделю 2 на реальном коде.

rsgrep принимает шаблон и пути к файлам, печатает совпадающие строки с номерами и именем файла. На этой неделе требуется каркас: разбор аргументов вручную (clap придёт позже), чтение файла в `String`, проход по строкам и наивный поиск подстроки. Всё вокруг поиска строится на владении: `String` владеет содержимым, `&str` — окно в него; ошибки чтения возвращаются через `Result` (подробно — неделя 4, пока достаточно `?`).

SRC/MAIN.RS · КАРКАС НЕДЕЛИ 2

```rust
use std::{env, fs};

fn main() {
    let args: Vec<String> = env::args().collect();
    if args.len() < 3 {
        eprintln!("usage: rsgrep PATTERN FILE...");
        std::process::exit(2);
    }
    let pattern = &args[1];
    for path in &args[2..] {
        let content = match fs::read_to_string(path) {
            Ok(c) => c,
            Err(e) => { eprintln!("rsgrep: {path}: {e}"); continue; }
        };
        for (i, line) in content.lines().enumerate() {
            if line.contains(pattern) {
                println!("{path}:{}: {line}", i + 1);
            }
        }
```

```
    }
}
```

ЧЕК-ТОЧКА НЕДЕЛИ

`cargo run -- rust src/main.rs` печатает строки с «rust». Коммит с осмысленным сообщением; в README — как собрать и запустить.

ДОМАШКА Добавить флаг `-n` (нумерация строк) и `-v` (инверсия поиска), парсить их вручную; написать 3 smoke-теста на собственном файле-фикстуре.

НЕДЕЛЯ 3 Данные: строки, коллекции, структуры, enum

УРОК 1.15 25 мин

String и &str: UTF-8 насквозь

ЦЕЛЬ — понять, почему у String нет индексации по `s[3]` и как правильно ходить по строкам.

`String` — владеющая, растущая UTF-8-строка (тройка ptr/len/capacity, как `std::string` C++, но гарантированно UTF-8); `&str` — заимствованное окно. Индексация по байту запрещена: граница может пройти посередине многобайтового символа, и вместо «символа» получится паника или невалидная строка. Правильные способы — итерация `.chars()` по юникод-скалярам, `.bytes()` по байтам и срезы только по границам графем, которые возвращает `char_indices()`. Конкатенация: `+` переиспользует буфер первого String (забирая владение), `format!` удобнее для многих частей.

```
fn main() {
    let s = String::from("привет, мир");
    // let c = s[3];                      // ошибка: String не индексируется
    let chars: Vec<char> = s.chars().collect();
    println!("{} {}", chars.len(), s.len()); // 11 символов, 20 байт

    for (i, ch) in "ёжик".char_indices() {
        if i > 0 { print!(" "); }
        print!("{i}:{ch}");
    }
    println!();

    let mut s2 = String::with_capacity(64);  // избежать реаллокаций
    s2.push_str("rlog-");
    s2.push('x');
    let s3 = format!("{s2}-{s}-ok");          // новая строка из кусков
    println!("{s3}");
}
```

СРАВНЕНИЕ

Go: `string` — read-only UTF-8, `[]rune` — способ ходить по кодовым точкам; Rust: `String`/`&str` разделяют владение/заём так же, как `[]byte`/string, но с гарантией валидности UTF-8. C++: `std::string` — просто байты; кодировка — ваша ответственность.

ДОМАШКА Функция `truncate_chars(s: &str, n: usize) -> String` — обрезка по символам без паник на UTF-8; тесты на смайлики и кириллицу.

УРОК 1.16 25 мин

Vec<T>: рост, capacity, паттерны доступа

ЦЕЛЬ — уверенно работать с динамическим массивом и понимать цену операций.

`Vec<T>` — непрерывный буфер в куче с амортизированным push за O(1): при переполнении capacity удваивается и происходит переезд — то же, что `std::vector` и Go-срез, только рост детерминирован удвоением. Доступ по индексу паникует при выходе за границы (в debug — точно, в release — тоже, это не UB, а безопасная паника); метод `get` возвращает `Option`. Удаление из середины — O(n) через `remove`, через `swap_remove` — O(1) ценой перемены порядка. Важный паттерн: собрать в Vec индексы, а потом мутировать — обход конфликтов заёмов из урока 1.11.

```
fn dedup_adjacent(v: &mut Vec<i32>) {
    if v.is_empty() { return; }
    let mut w = 1;                          // указатель записи
    for i in 1..v.len() {
        if v[i] ≠ v[w - 1] { v[w] = v[i]; w += 1; }
    }
    v.truncate(w);                          // безопасно: заём уже закончился
}

fn main() {
    let mut v = Vec::with_capacity(8);
    v.extend([1, 1, 2, 3, 3, 3]);
    dedup_adjacent(&mut v);
    println!("{v:?}");                      // [1, 2, 3]

    match v.get(10) {
        Some(x) ⇒ println!("{x}"),
        None ⇒ println!("нет элемента, но без паники"),
    }
}
```

> СРАВНЕНИЕ
> Go: `append` может вернуть новый backing array — указатели, взятые до append, молча показывают старые данные. C++: `push_back` инвалидирует итераторы, и это UB. Rust: инвалидация — ошибка компиляции.

ДОМАШКА Реализовать `matrix_transpose(rows: Vec<Vec<i32>>) -> Vec<Vec<i32>>` и `running_sum(&mut Vec<i32>)`; тесты + clippy-чистота.

УРОК 1.17 25 мин

HashMap/HashSet: hasher и entry API

ЦЕЛЬ — освоить словари, entry-паттерн и осознанный выбор хешера.

Стандартный `HashMap<K, V>` защищён от hash-flooding случайным сидом SipHash — это безопасно, но медленнее, чем простые хешеры; в hot path ставят `FxHasher` или `ahash` (крейты `rustc-hash`, `ahash`), в embedded-контексте иногда вообще `BTreeMap` без хешера. Главный идиоматический инструмент — `entry()`: атомарное «получи или создай», которое закрывает типичный Go-паттерн двух обращений к map. Ключи обязаны быть `Eq + Hash` — derive их даёт почти всегда.

```
use std::collections::HashMap;

fn main() {
```

```
    for ch in text.chars().filter(|c| !c.is_whitespace()) {
        *freq.entry(ch).or_insert(0) += 1;      // get-or-insert одной операцией
    }
    let top = freq.iter().max_by_key(|(_, n)| **n).unwrap();
    println!("топ: '{}' x{}", top.0, top.1);

    // Go-стиль "если нет — вычисли и положи" без двойного хеширования:letmut cache: HashMap<u32,
String> = HashMap::new();
    let v = cache.entry(42).or_insert_with(|| format!("value-{42}"));
    println!("{v}");
}
```

СРАВНЕНИЕ

Go: `v, ok := m[k]; if !ok { m[k] = ... }` — два хеширования и гонка при конкурентном доступе. C++: `emplace`/`try_emplace` близко по духу, но verbose. Rust: entry — читается как английский.

ДОМАШКА Частотный словарь слов файла с выводом топ-5; ключ — `String`, используйте entry; сравните время с `FxHashMap` (крейт rustc-hash) на файле ~10 МБ.

УРОК 1.18 25 мин

Структуры и методы: impl, self, конструкторы

ЦЕЛЬ — понять три формы self и как строить API структур без конструкторов в C++-смысле.

Методы живут в блоке `impl`. Три формы приёмника: `&self` — читаем; `&mut self` — меняем; `self` — потребляем (владение переходит, объект после вызова мёртв). Конструктор — это просто связанная функция `new` без self; удобные дефолты даёт derive `Default`. Ассоциированные константы и функции — аналог статических членов. Отдельный рудимент стиля: один тип — один impl-блок по смыслу (тут конструкторы, там тесты), разрывов инкапсуляции, как private/public в C++, тут нет — есть только `pub` для модулей (урок 1.22).

```
#[derive(Debug, Default, Clone)]
pub struct Ring {
    buf: Vec<i32>,
    head: usize,
    len: usize,
}

impl Ring {
    pub fn with_capacity(cap: usize) -> Self {    // связанная функция-«конструктор»
        Ring { buf: vec![0; cap], head: 0, len: 0 }
    }
    pub fn push(&mut self, v: i32) {              // &mut self — writer
        let tail = (self.head + self.len) % self.buf.len();
        self.buf[tail] = v;
        if self.len == self.buf.len() { self.head = (self.head + 1) % self.buf.len(); }
        else { self.len += 1; }
    }
    pub fn peek(&self) -> Option<i32> {           // &self — reader
        if self.len == 0 { return None; }
        Some(self.buf[self.head])
    }
}

fn main() {
    let mut r = Ring::with_capacity(3);
    r.push(1); r.push(2); r.push(3); r.push(4);   // перезапишет 1
```

```
println!("{:?} peek={:?}", r, r.peek());
}
```

СРАВНЕНИЕ

Go: методы с value/pointer receiver — копия или указатель, компилятор не различает строго. C++: const-методы — соглашение, конструкторы/деструкторы — сложная семантика. Rust: три self-формы — это ровно три режима владения.

ДОМАШКА Доработать Ring: метод `pop(&mut self) -> Option<i32>` и `iter(&self) -> impl Iterator<Item = i32>`; юнит-тесты на переполнение и пустой буфер.

УРОК 1.19 25 мин

Option: отсутствие значения без null

ЦЕЛЬ — переключиться с null-проверок на Option-комбинаторы и перестать разворачивать всё через unwrap.

`Option<T>` — enum из `Some(T)` и `None`: отсутствие значения — часть типа, а не сюрприз в рантайме. «Миллиард-долларовая ошибка» null здесь решена структурно. Комбинаторы (`map`, `and_then`, `unwrap_or`, `ok_or`) позволяют строить цепочки без вложенных if. `unwrap()` — для тестов и прототипов; в продакшен-коде — `?`, `expect("причина")` или проброс. В API это переносит ответственность на компилятор: функция, возвращающая `Option<User>`, физически не может «забыть» сказать, что пользователя нет.

```
struct User { name: String, email: Option<String> }

fn contact_email(u: &User, fallback: &str) -> String {
    u.email.clone()
        .filter(|e| e.contains('@'))            // валидация на лету
        .unwrap_or_else(|| fallback.to_owned())
}

fn find_user(db: &[User], name: &str) -> Option<&User> {
    db.iter().find(|u| u.name == name)
}

fn main() {
    let db = vec![
        User { name: "ada".into(), email: Some("ada@lovelace.dev".into()) },
        User { name: "bob".into(), email: None },
    ];
    let u = find_user(&db, "bob").ok_or("нет пользователя bob");  // Result!
    println!("{}", contact_email(u.unwrap(), "ops@corp.io"));
}
```

СРАВНЕНИЕ

Go: `v, ok :=` — два значения и забываемая проверка; nil-pointer panic — routine. C++: `std::optional` — близкий аналог, но без `?` и с пониженной тягой к комбинаторам в реальном коде.

ДОМАШКА Переписать 5 функций из прошлых ДЗ, где возвращался `-1` или `false`, на `Option`; убрать из них все `unwrap()` кроме тестов.

УРОК 1.20 25 мин

Дизайн данных: композиция, newtype, состояние через enum

ЦЕЛЬ — собирать типы так, чтобы невалидные состояния не выражались, и переносить ООП-навыки на композицию.

Наследования в Rust нет — есть композиция и трейты (неделя 5). Это не ограничение, а направление: расширять поведение — трейтами, переиспользовать данные — вложением. Два рабочих паттерна. Newtype: обёртка `struct Meters(f64)` делает единицы измерения частью типов и бесплатно получает методы через `impl`. Моделирование состояния: вместо bool-флагов `is_connected, is_authenticated, is_premium` — один enum, каждая ветка match получает только допустимые для состояния поля. Невалидное сочетание флагов в такой модели не выражается синтаксически.

```
struct Meters(f64);
struct Seconds(f64);

struct Speed(f64);                          // результат — тоже тип
fn div(a: Meters, b: Seconds) -> Speed { Speed(a.0 / b.0) }
// div(a, Meters(5.0)) — не скомпилируется: единицы не спутаешь

enum Conn {                                  // вместо 3 bool
    Disconnected,
    Connected { addr: String },
    Authenticated { addr: String, token: String },
}

fn send(conn: &Conn, msg: &str) -> Result<(), String> {
    match conn {
        Conn::Authenticated { token, .. } => {
            println!("отправляю с токеном {token}: {msg}");
            Ok(())
        }
        _ => Err("нет аутентификации".into()),
    }
}
```

> СРАВНЕНИЕ
> C++/Go-код с флагами `if connected && authenticated && !closed` — классический источник багов о невалидных состояниях; типовая модель делает такие состояния нецелыми (unrepresentable).

ДОМАШКА Смоделировать банкомат: enum состояний (Idle, CardIn, PinOk, Session) с данными в вариантах; функции `insert_card`, `enter_pin`, `withdraw` принимают состояние и возвращают `Result<Conn, AtmError>`. Тесты на недопустимые переходы.

УРОК 1.21 20 мин

Лаба 1 · часть 2: поиск через match и структуры

ЦЕЛЬ — превратить каркас rsgrep в структурированную программу: отдельные модули, тип Config, словарь по строкам.

На этой неделе каркас получает форму: конфигурация (шаблон, флаги, пути) собирается в структуру `Config` с методом `parse`; поиск выносится в функцию, возвращающую `Vec<Match>`; строки файлов обрабатываются без лишних аллокаций — всё через `&str`.

Требование недели: `cargo clippy` чист, история коммитов читаемая (по коммиту на осмысленный шаг).

SRC/CONFIG.RS

```rust
pub struct Config {
    pub pattern: String,
    pub invert: bool,
    pub files: Vec<String>,
}

impl Config {
    pub fn parse(args: &[String]) -> Result<Config, String> {
        let mut flags = args.iter().filter(|a| a.starts_with('-'));
        let rest: Vec<String> = args.iter()
            .filter(|a| !a.starts_with('-'))
            .cloned().collect();
        let [pattern, files @ ..] = rest.as_slice() else {
            return Err("нужны PATTERN и хотя бы один FILE".into());
        };
        Ok(Config {
            pattern: pattern.clone(),
            invert: flags.any(|f| f == "-v"),
            files: files.to_vec(),
        })
    }
}
```

ЧЕК-ТОЧКА НЕДЕЛИ

Config вынесен в модуль, let-else разобран, тесты на parse покрывают ошибку «нет аргументов». Коммит + тег lab1-part2.

ДОМАШКА Добавить в rsgrep флаг `-c` (только счётчик совпадений на файл) и флаг `-i` (без учёта регистра; посмотрите `eq_ignore_ascii_case` и `to_lowercase` — обсудите цену аллокации).

НЕДЕЛЯ 4 Ошибки, модули, тестирование — и сдача лабы 1

УРОК 1.22 25 мин

Паника против ошибки: два механизма

ЦЕЛЬ — научиться выбирать между `panic!` и `Result` по осмысленному критерию.

В Rust два канала сбоев. Паника — разворачивание стека с прерыванием потока: путь для багов программиста (нарушение инварианта, невозможное состояние, «туда мы не должны были попасть»). `Result` — типизированный канал для ожидаемых сбоев: файл не найден, сеть отвалилась, ввод кривой. В embedded-контексте (модуль 7) паника по умолчанию означает «зависание до ресета», поэтому политика паник там строже. Практическое правило: если вызывающий может что-то сделать с ошибкой — это Result; если нет — паника с внятным сообщением. Методы-суффиксы: `unwrap` паникует, `expect` паникует с вашей формулировкой, `unwrap_or(_else)` подставляет дефолт.

```rust
fn parse_port(s: &str) -> Result<u16, String> {
    let n: u32 = s.parse().map_err(|e| format!("не число: {e}"))?;
    if n == 0 || n > 65535 {
        return Err(format!("порт {n} вне диапазона"));
    }
    Ok(n as u16)
```

```rust
}

fn main() {
    match parse_port("8080") {
        Ok(p)  => println!("порт {p}"),
        Err(e) => eprintln!("ошибка конфигурации: {e}"),
    }
    let known = ["http", "https"];
    let port = known.iter().position(|&s| s == "grpc")
        .expect("внутренний инвариант: протокол есть в таблице"); // баг, не данные
}
```

СРАВНЕНИЕ

Go: `panic`/`recover` — экзотика для багов, `error` — основной путь: философия совпадает, но Result статически навязывает проверку. C++: исключения смешивают оба канала; `std::expected` (C++23) — движение в сторону Rust.

ДОМАШКА Провести аудит своего rsgrep: найти все `unwrap/expect/panic` и классифицировать — баг программиста или ожидаемый сбой; заменить вторые на Result-путь.

УРОК 1.23 25 мин

Result и оператор ?

ЦЕЛЬ — освоить «?» как механизм раннего выхода и комбинаторы для коротких цепочек.

`Result<T, E>` — enum `Ok(T) / Err(E)`. Оператор `?` — это «верни ошибку вверх, если она есть»: работает только в функциях, возвращающих Result с совместимым типом ошибки, и с конверсией через трейт `From`. Комбинаторы закрывают типовые сценарии: `map` — преобразовать успех, `map_err` — ошибку, `and_then` — цепочку с последующим Result, `unwrap_or_default` — дефолт. Соответствие Go: `if err ≠ nil { return err }` — это и есть `?`, только свёрнутое в один символ и с автоматической конверсией типов.

```rust
use std::fs;
use std::num::ParseIntError;

fn read_number(path: &str) -> Result<i32, String> {
    // map_err приводит io::Error к общему типу, затем "?" делает ранний выход
    let text = fs::read_to_string(path)
        .map_err(|e| format!("не удалось прочитать {path}: {e}"))?;
    let n: i32 = text.trim().parse()
        .map_err(|e: ParseIntError| format!("не число в {path}: {e}"))?;
    Ok(n * 2)
}

fn main() {
    // "?" в выражении: Result внутри выражения удобно разворачивать через match
    let n = match read_number("config.txt") {
        Ok(v) => v.clamp(0, 1000),
        Err(e) => { eprintln!("{e}"); 0 }
    };
    println!("n = {n}");

    // parse выводит тип ошибки сам — сигнатуру писать не обязательно:
    let parsed: Result<i32, ParseIntError> = "42".parse();
    assert_eq!(parsed.unwrap(), 42);
}
```

ПРАКТИКА Написать цепочку: прочитать файл → распарсить CSV-строку → извлечь число, используя только `?` и `map_err`; посчитать, сколько строк кода сэкономил «?» против match-версии.

ДОМАШКА В rsgrep: чтение файлов перевести на функцию `run(&Config) -> Result<Vec<Match>, GrepError>` без единого panic в lib-части.

УРОК 1.24 25 мин

Свои типы ошибок: enum ошибок, thiserror/anyhow

ЦЕЛЬ — проектировать enum ошибок библиотеки и понимать, когда хватает anyhow.

Тип ошибки — такой же тип, как и остальные: для библиотеки — публичный enum с вариантами по смыслу, реализующий `Display` и `Error`; конверсии из чужих ошибок — через `impl From`, после чего «?» конвертирует автоматически. Для приложений, где классификация не нужна, используют крейт `anyhow` (динамическая ошибка с контекстом), для библиотек — `thiserror` (макрос, генерирующий Display/From). Правило разделения: библиотека отдаёт структурные ошибки, приложение решает, что с ними делать.

```
use std::fmt;

#[derive(Debug)]
pub enum GrepError {
    Io(std::io::Error),
    BadPattern(String),
    BinaryFile(String),
}

impl fmt::Display for GrepError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            GrepError::Io(e)          => write!(f, "ошибка ввода-вывода: {e}"),
            GrepError::BadPattern(p)  => write!(f, "неверный шаблон: {p}"),
            GrepError::BinaryFile(p)  => write!(f, "пропущен бинарный файл: {p}"),
        }
    }
}

impl std::error::Error for GrepError {}
impl From<std::io::Error> for GrepError {
    fn from(e: std::io::Error) -> Self { GrepError::Io(e) }
}
// теперь "?" из io::Error даёт GrepError::Io автоматически
```

СРАВНЕНИЕ

Go: `errors.Is/As` по sentinel-ошибкам и кастомным типам; Rust-enum ошибок — то же самое, но с исчерпывающим match. thiserror ≈ генерация boilerplate, который в Go пишут руками.

ДОМАШКА Ввести `GrepError` в rsgrep; добавить вариант `EmptyPattern` и обработку в main: код выхода 1 для Io, 2 для BadPattern. Сравнить версию с thiserror (добавить крейт) и вручную — оставить одну.

УРОК 1.25 20 мин

Модули: mod, pub, use, дерево крейта

ЦЕЛЬ — управлять видимостью и структурой проекта любого размера.

Крейт компилируется из дерева модулей, корень — `src/main.rs` (бинарь) или `src/lib.rs` (библиотека). Всё по умолчанию приватно; `pub` открывает элемент наверх, `pub(crate)` —

только внутри крейта. `mod file;` подключает `file.rs` (или `file/mod.rs`); `use` создаёт локальный путь; `super::` и `crate::` — навигация вверх и к корню. Приватность в Rust работает на уровне модулей, а не классов: тестовый модуль видит приватные поля своего крейта — и это осознанно.

СТРУКТУРА RSGREP

```
src/
  main.rs        // CLI-обвязка, exit-коды
  lib.rs         // pub mod config; pub mod search;
  config.rs      // pub struct Config + parse
  search.rs      // pub fn run(...) -> Result<Vec<Match>, GrepError>

<span class="cap">src/lib.rs</span>
pub mod config;
pub mod search;
mod internal;       // приватный: виден только внутри крейта

pub use search::run;   // реэкспорт: пользователю хватит rsgrep::run
```

> **СРАВНЕНИЕ**
> Go: имя с заглавной буквы = exported, мелочь, но правила глобальные. C++: заголовки + инклюд-гарды + ODR-грабли. Rust: модули и порядок компиляции описаны декларативно, заголовков нет вообще.

ДОМАШКА Разложить rsgrep на модули по схеме выше; main.rs не должен содержать ничего, кроме аргументов, вызова lib и печати результата.

УРОК 1.26 25 мин

Тестирование: unit, integration, doc-тесты

ЦЕЛЬ — встроить тесты в рабочий цикл: cargo test, покрытие краёв, фикстуры.

Unit-тесты живут рядом с кодом в модуле `#[cfg(test)]` и видят приватные элементы. Интеграционные — в каталоге `tests/`, по файлу на крейт-клиент: они тестируют только публичный API. Doc-тесты в `///`-комментариях исполняются при `cargo test` — документация не врёт. Полезные инструменты: `assert_eq!`, `assert!`, `#[should_panic]`, `Result` в тесте для ошибки с «?»; фильтр запуска — `cargo test имя`. Тест как один из критериев приёмки лаб: не проходит тест — лаба не сдана.

```rust
pub fn add(a: i64, b: i64) -> i64 { a + b }

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn basics() { assert_eq!(add(2, 3), 5); }

    #[test]
    fn overflow_is_wrapping_in_release() {
        assert_eq!(add(i64::MAX, 1), i64::MIN); //debug: panic, release: wrap
    }

    #[test]
    fn err_path() -> Result<(), String> {
        let v: Vec<i32> = "1,2".split(',').map(|s| s.parse().map_err(|e: _| e.to_string()))
            .collect::<Result<_, _>>()?;
        assert_eq!(v, vec![1, 2]);
        Ok(())
```

```
    }
}
```

СРАВНЕНИЕ

Go: `_test.go` — тот же принцип; Rust добавляет doc-тесты и встроенный прогон по фильтру. C++: обычно отдельный фреймворк (gtest/catch) и ручная интеграция в сборку.

ДОМАШКА Довести покрытие rsgrep: тесты на Config.parse (4 случая), на search.run (фикстура-файл в tests/data), один doc-тест на run с примером в ///.

УРОК 1.27 20 мин

Лаба 1 · сдача и код-ревью

ЦЕЛЬ — закрыть лабу №1 и пройти первую формальную приёмку.

rsgrep к сдаче: модульная структура (config/search/lib), флаги -n -v -c -i, тип GrepError с обработкой exit-кодов, тесты, README с примерами, чистые clippy и fmt. Код-ревью идёт по чек-листу: нет unwrap вне тестов; функции принимают &str/&[T], а не String/Vec, когда владение не нужно; ошибки структурные; история коммитов отражает шаги разработки. Типовые замечания первой ревью-сессии стоит записать — они же станут критериями самопроверки всех последующих лаб.

ЛАБОРАТОРНАЯ РАБОТА №1 · 8% ИТОГОВОЙ ОЦЕНКИ

rsgrep — поиск по файлам

Минимум для зачёта: сборка без warnings; поиск по нескольким файлам; флаги -n/-v/-c/-i; структурные ошибки; unit + интеграционные тесты; README.

- **Базовый уровень:** наивный substring-поиск, корректные exit-коды (0/1/2).
- **Продвинутый:** потоковое чтение больших файлов (BufReader), поиск в stdin при «-» вместо пути.
- **Со звёздочкой:** регэкспы через крейт regex и бенчмарк против ripgrep на файле 100 МБ (просто таблица чисел).

ДОМАШКА Устранить замечания ревью; написать diff-обзор «что изменилось после ревью и почему» — 10 строк в PR-описании.

УРОК 1.28 25 мин

Синтез недели: краевые случаи и повторение

ЦЕЛЬ — закрепить модуль разбором «боевого» примера: чтение конфига с дефолтами, где работает всё изученное сразу.

Завершающее занятие модуля — сборка мини-задачи, объединяющая владение, Option/Result, enum-состояния, модули и тесты. Конфиг из файла: прочитать, распарсить «ключ=значение», применить дефолты, провалидировать. Каждый шаг — отдельная функция с честной сигнатурой; ошибки — свой enum; состояние — типы-обёртки. После разбора — самостоятельная проверка по чек-листу модуля и план на модуль 2.

```rust
use std::collections::HashMap;

#[derive(Debug)]
pub struct Settings { pub port: u16, pub threads: u8, pub verbose: bool }

pub fn load(text: &str) -> Result<Settings, String> {
```

```
    letmut kv: HashMap<String, String> = HashMap::new();
    for (i, line) in text.lines().map(str::trim).enumerate() {
        if line.is_empty() || line.starts_with('#') { continue; }
        let (k, v) = line.split_once('=')
            .ok_or_else(|| format!("строка {}: нет '='", i + 1))?;
        kv.insert(k.trim().into(), v.trim().into());
    }
    Ok(Settings {
        port: kv.get("port").map(|v| v.parse()).transpose()
                .map_err(|e: _| format!("port: {e}"))?.unwrap_or(8080),
        threads: 4,
        verbose: kv.get("verbose").map(|v| v == "true").unwrap_or(false),
    })
}

#[test]fn defaults_and_errors() {
    assert_eq!(load("").unwrap().port, 8080);
    assert!(load("port=abc").is_err());
}
```

РИТМ ЗАКРЕПЛЕНИЯ

Одно «синтез-занятие» будет закрывать каждый модуль: задача-сборка без новых концепций, только связывание уже изученных. Это точка, где знание превращается в навык.

ДОМАШКА Прогнать чек-лист модуля 1 ниже; слабые пункты — отметить и закрыть на повторении перед модулем 2. Сдать ссылку на репозиторий rsgrep.

КОНТРОЛЬ

Чек-лист модуля 1: умею / написал / объясню

- ☐ **Объясняю** move/Copy/Clone и предсказываю, где будет E0382, ещё до компиляции.
- ☐ **Объясняю** правила &/&mut и читаю E0502 как «у меня одновременно писатель и читатель».
- ☐ **Объясняю** lifetime-аннотации как связи, знаю три правила elision.
- ☐ **Написал** Ring с тремя формами self и итератором.
- ☐ **Написал** rsgrep: модули, GrepError, флаги, тесты, README.
- ☐ **Написал** ≥ 20 unit-тестов суммарно по ДЗ и лабе.
- ☐ **Умею** без гугла: Vec/String/HashMap entry, Option/Result комбинаторы, «?».
- ☐ **Умею** моделировать состояния enum-ом так, что невалидные не выражаются.

МОДУЛЬ 2 · НЕДЕЛИ 5–6 · ЗАНЯТИЯ 2.1–2.10

Система типов в полную силу

2 недели · 10 занятий · ≈ 4 ч теории · трейты · генерики · итераторы · умные указатели

Модуль 2 отвечает на вопрос «как Rust добивается полиморфизма без наследования и без виртуальных функций там, где они не нужны». Трейты здесь — не просто интерфейсы: это контракты с методами по умолчанию, ассоциированными типами и генерик-границами, разрешаемые либо мономорфизацией (быстро, как шаблоны C++), либо через трейт-объекты (динамически, как интерфейсы Go). Вторая половина модуля — функциональный слой:

замыкания и итераторы, на которых строится идиоматический стиль обработки данных, и умные указатели `Box/Rc/RefCell/Arc`, которые нужны ровно тогда, когда строгость владения мешает выразить графовые структуры. После этого модуля код перестаёт выглядеть «как C++ с borrow checker» и начинает выглядеть по-растамански.

НЕДЕЛЯ 5 Трейты и генерики

УРОК 2.1 25 мин

Трейты: контракт без наследования

ЦЕЛЬ — определить и реализовать трейт, понять методы по умолчанию и связь с интерфейсами Go.

Трейт описывает поведение: набор методов, которые обязан иметь реализующий тип. Реализация — явная: `impl Trait for Type`, никаких неявных структурных интерфейсов, как в Go. Дефолтные реализации позволяют трейту давать «бесплатные» методы, выводимые из обязательных — так устроен `Iterator`: реализуете один `next`, получаете десятки адаптеров. Трейт-границы в сигнатурах (`fn f(x: &impl Summary)`) говорят «принимаю любой тип с этим поведением». Правило осмотрительности, унаследованное из Go: много маленьких трейтов лучше одного жирного.

```
trait Summary {
    fn author(&self) -> String;                 // обязательный метод
    fn preview(&self) -> String {               // дефолтный метод
        format!("(автор: {})", self.author())
    }
}

struct Article { title: String, author: String }

impl Summary for Article {
    fn author(&self) -> String { self.author.clone() }
}

struct Tweet { user: String }

impl Summary for Tweet {
    fn author(&self) -> String { format!("@{}", self.user) }
    fn preview(&self) -> String { format!("тред от {}", self.author()) }
}

fn print_banner(item: &impl Summary) {          // генерик-сокращение
    println!("== {} ==", item.preview());
}
```

СРАВНЕНИЕ

Go: интерфейсы неявные — тип удовлетворяет интерфейс автоматически; Rust требует явного impl, зато виден полный список контрактов и работает утиная типизация через генерики без рантайм-стоимости. C++20 concepts — ближайший аналог трейт-границ.

ДОМАШКА Трейт `Metric` с методом `value(&self) -> f64` и дефолтным `report`; реализовать для `Temperature`, `Pressure`, `Humidity`; функция `worst(&[&impl Metric])`.

УРОК 2.2 25 мин

Генерики: мономорфизация против стирания типов

ЦЕЛЬ — понять, во что превращается `<T>` после компиляции, и писать генерик-функции без страха.

Генерик в Rust — не шаблон-текст, как в C++, и не интерфейс, как в Go: компилятор выполняет мономорфизацию — для каждого конкретного типа генерируется специализированная копия функции. Ноль накладных расходов, как у шаблонов C++, но с проверкой корректности до инстанцирования: тело должно быть валидно для любого T, удовлетворяющего границам. Отсюда важное следствие: в генерике доступны только методы трейтов из границ, а не «всё подряд» — это дисциплинирует контракт.

```
fn largest<T: PartialOrd>(list: &[T]) -> Option<&T> {
    list.iter().reduce(|a, b| if a >= b { a } else { b })
}

fn count_matches<T>(list: &[T], pred: impl Fn(&T) -> bool) -> usize {
    list.iter().filter(|t| pred(t)).count()
}

fn main() {
    let nums = vec![3, 7, 2, 9];
    let strs = vec!["ada", "grace", "linus"];
    println!("{:?} {:?}", largest(&nums), largest(&strs));
    println!("{}", count_matches(&nums, |&x| x % 2 == 0));
}
```

> СРАВНЕНИЕ
> C++-шаблоны инстанцируются с диагностикой в момент использования (ошибки на сотни строк); Rust-генерики проверяются по границам сразу. Go-дженерики (1.18+) тоже мономорфизируются, но словари для методов ограничивают производительность; Rust-границы — чистые требования компиляции.

ДОМАШКА Написать генерик-функции `min_by_key`, `is_sorted`, `chunk_sums`; протестировать на i32, f64, String; объяснить в комментарии, какие трейт-границы каждой нужны и почему.

УРОК 2.3 30 мин

Трейт-объекты: dyn Trait и когда динамический полиморфизм оправдан

ЦЕЛЬ — освоить `dyn Trait`, object safety и критерий выбора между статической и динамической диспетчеризацией.

`dyn Trait` — тип «любое значение, реализующее трейт», живущее за vtable-указателем: толстый указатель (данные + vtable) размера двух слов. Это динамическая диспетчеризация: один вызов по указателю функции, как виртуальный метод C++ или интерфейс Go. Требование object safety: методы не должны возвращать Self или иметь генерик-параметры. Правило выбора: по умолчанию — генерики (нулевая цена, инлайн); dyn — когда нужен гетерогенный набор в одной коллекции, плагинная система или желание ускорить компиляцию, убрав взрыв мономорфизации.

```
trait Sensor {
    fn read(&self) -> f64;
    fn name(&self) -> &str;
}

struct Thermometer { celsius: f64 }
struct Voltmeter  { volts: f64 }
```

```
impl Sensor for Thermometer {
    fn read(&self) -> f64 { self.celsius }
    fn name(&self) -> &str { "th" }
}
impl Sensor for Voltmeter {
    fn read(&self) -> f64 { self.volts }
    fn name(&self) -> &str { "vm" }
}

fn main() {
    // гетерогенная коллекция: генерик тут не поможет, нужен dyn
    let bus: Vec<Box<dyn Sensor>> = vec![
        Box::new(Thermometer { celsius: 36.6 }),
        Box::new(Voltmeter { volts: 3.3 }),
    ];
    for s in &bus {
        println!("{}: {:.1}", s.name(), s.read());   // vtable-вызов
    }
}
```

> СРАВНЕНИЕ
>
> Go: `[]Sensor` интерфейсов — это и есть Vec<Box<dyn Sensor>>, просто слово Box скрыто рантаймом. C++: виртуальные методы + unique_ptr<Base>. Rust даёт обе модели осознанно: статическую (генерик) и динамическую (dyn).

ДОМАШКА Расширить Metric из ДЗ 2.1 до гетерогенного дашборда: `Vec<Box<dyn Metric>>`, вывод таблицы; затем заменить на генерик-версию и замерить размер бинарей обоих вариантов (`cargo build --release`, размер файла).

УРОК 2.4 25 мин

Стандартные трейты, которые надо знать наизусть

ЦЕЛЬ — собрать «джентльменский набор» трейтов std и понимать, что даёт каждый derive.

Библиотека std построена на небольшом наборе ключевых трейтов, и знание их — это знание самого языка. `Debug` (печать {:?}) и `Display` ({}), `Clone`/`Copy`, `PartialEq`/`Eq` и `PartialOrd`/`Ord` (сортировка и ключи карт), `Default`, `Hash` (ключи HashMap), `From`/`Into` (конверсии), `Drop`, `Iterator`, `Send`/`Sync` (потокобезопасность, модуль 4). Большинство доступны через `#[derive(...)]`, но понимать, что стоит за derive, обязан каждый: например, Eq невозможен для f64 (нет total order из-за NaN), и HashMap с ключом-f64 просто не скомпилируется.

```
#[derive(Debug, Clone, PartialEq, Eq, Hash, Default)]
struct Point { x: i32, y: i32 }

impl std::fmt::Display for Point {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "({}, {})", self.x, self.y)     // Display руками: формат — ваш выбор
    }
}

impl From<(i32, i32)> for Point {
    fn from(t: (i32, i32)) -> Self { Point { x: t.0, y: t.1 } }
}

fn main() {
    let p = Point::from((3, 4));
    let set: std::collections::HashSet<Point> = [p.clone(), Point::default()].into();
```

```
    println!("{p} set={set:?} eq={}", p == Point { x: 3, y: 4 });
}
```

> **МИНИ-ШПАРГАЛКА**
>
> Ключ HashMap → Eq + Hash. Сортировка → Ord. Печать разработчику → Debug, пользователю → Display. Конверсия типов в своём API → From (Into получите бесплатно).

ДОМАШКА Тип `Version(major, minor, patch)`: derive-набор под задачи «ключ в HashMap», «сортировка», «вывод как 1.2.3» (Display руками); тесты порядка 1.9 > 1.10 по semver-логике.

УРОК 2.5 25 мин

Ассоциированные типы, where-клаузы, границы в действии

ЦЕЛЬ — читать сложные сигнатуры std без страха и писать свои с ассоциированными типами.

Ассоциированный тип — «плейсхолдер, который каждая реализация фиксирует один раз»: `trait Iterator { type Item; fn next(&mut self) -> Option<Self::Item>; }`. Это снимает шум генерик-параметров: у итератора ровно один Item на реализацию, а не тип-параметр на каждый вызов. Генерик-параметры нужны, когда один тип может реализовать трейт с разными параметрами (например, From<T> для многих T). Where-клаузы — синтаксис для сложных границ; в std они везде, и умение их читать — часть беглости.

```
trait Container {
    type Item;
    fn get(&self, i: usize) -> Option<&Self::Item>;
    fn len(&self) -> usize;
}

impl<T> Container for Vec<T> {
    type Item = T;
    fn get(&self, i: usize) -> Option<&T> { self.get(i) }
    fn len(&self) -> usize { Vec::len(self) }
}

// where-версия читабельна при длинных границах:
fn first_or_default<C>(c: &C) -> Option<String>
where
    C: Container,
    C::Item: std::fmt::Display,
{
    c.get(0).map(|x| x.to_string())
}
```

ПРАКТИКА Реализовать трейт `Cache` с ассоциированным типом `Key` и методами get/put для двух хранилищ: HashMap и Vec (линейный поиск); один общий тестовый набор для обоих.

ДОМАШКА Написать функцию `top_n<C>(c: &C, n: usize) -> Vec<String>` через where-границы: C — Container, Item — Display + Ord; тесты на Vec<i32> и Vec<String>.

НЕДЕЛЯ 6 **Замыкания, итераторы, умные указатели**

УРОК 2.6 25 мин

Замыкания: Fn, FnMut, FnOnce

ЦЕЛЬ — понять три трейта замыканий и как они связаны с владением захваченных переменных.

Замыкание — анонимная функция с захватом окружения. Компилятор выбирает режим захвата минимально необходимый, а три трейта описывают возможности вызова: `FnOnce` — можно вызвать один раз (потребляет захваченное, как move-замыкание с передачей владения), `FnMut` — много раз с мутацией, `Fn` — много раз только для чтения. Иерархия: любой Fn — это FnMut, любой FnMut — FnOnce. В сигнатурах API выбирают самый широкий допустимый: колбэк-фильтр — Fn, накопитель — FnMut, спавн потока — FnOnce + `move`. Это прямое продолжение недели 2: замыкание — обычное значение, владеющее захваченными данными.

```
fn apply_once<F: FnOnce(i32) -> i32>(f: F, x: i32) -> i32 { f(x) }
fn apply_mut<F: FnMut(i32) -> i32>(mut f: F) -> i32 { f(1) + f(2) }
fn apply_fn<F: Fn(i32) -> i32>(f: F) -> i32 { f(10) }

fn main() {
    let base = 100;
    let add_base = |x| x + base;            // Fn: читает base по ссылке

    let mut acc = 0;
    let mut accumulate = |x| { acc += x; acc };  // FnMut: мутирует acc
    let _ = apply_mut(&mut accumulate);          // нужен mut у переменной

    let owned = String::from("токен");
    let consume = move || { println!("{owned}"); };  // FnOnce: move забрал владение
    consume();
    // consume();                               // E0382: замыкание потреблено
    println!("{} {} {}", apply_fn(add_base), accumulate(5), base);
    let _ = apply_once(|x| x * 2, 21);
}
```

СРАВНЕНИЕ

Go: замыкания захватывают по ссылке всегда — источник классической ошибки «в for-loop захватывается переменная итерации»; Rust-захват побайтово типизирован. C++: `[=]/[&]` — приблизительный аналог, но без трейтов и без проверки повторного вызова.

ДОМАШКА Сделать функцию `make_counter() -> impl FnMut() -> u32`, возвращающую независимые счётчики; тест: три счётчика растут отдельно. Объяснить, почему возвращается impl FnMut, а не Fn.

УРОК 2.7 25 мин

Итераторы: адаптеры и ленивость

ЦЕЛЬ — заменить циклы итераторными цепочками и понять их нулевую стоимость.

Трейт `Iterator` — один метод `next` и около 70 адаптеров-методов с дефолтными реализациями. Итераторы ленивы: `map`/`filter` ничего не делают до потребляющего вызова (`collect`, `sum`, `for`). Компилятор разворачивает цепочки в тот же машинный код, что и ручной цикл (zero-cost abstraction), поэтому итераторы — не «функциональная роскошь», а

идиома по умолчанию. Ключевые адаптеры: `map`, `filter`, `filter_map`, `take`/`skip`, `zip`, `enumerate`, `flat_map`, `fold`, `any`/`all`, `max_by_key`.

```
fn main() {
    let sensor_log = ["12.5", "abc", "13.1", "", "14.0", "x"];

    // Java-стиль с ручным циклом:
    let mut sum = 0.0f64;
    for line in sensor_log {
        if let Ok(v) = line.parse::<f64>() { sum += v; }
    }

    // Идиоматичный Rust — та же логика, нулевая цена:
    let sum2: f64 = sensor_log.iter()
        .filter_map(|l| l.trim().parse().ok())
        .sum();
    assert_eq!(sum, sum2);

    let report: Vec<String> = sensor_log.iter()
        .enumerate()
        .filter(|(_, l)| l.parse::<f64>().is_ok())
        .map(|(i, l)| format!("строка {}: {}", i + 1, l))
        .collect();
    println!("{report:?}");
}
```

> СРАВНЕНИЕ
>
> Go: ручные циклы + ручной аккумулятор; generics 1.18 дали slices.Map/Filter, но без ленивой композиции. C++: ranges (C++20) — концептуально то же самое, синтаксически многословнее. Rust: итераторы — базовый стиль, цикл — осознанное исключение.

ДОМАШКА Пайплайн обработки логов: из файла выбрать строки уровня ERROR, вытащить длительности (мс), посчитать p50/p95 вручную через итераторы (сортировка + индекс), вывести отчёт. Ноль `let mut` вне сортировки — попробуйте.

УРОК 2.8 25 мин

Свой итератор: реализуем Iterator с нуля

ЦЕЛЬ — написать тип, поддерживающий весь набор адаптеров, реализацией одного next.

Реализовать `Iterator` для своего типа — значит открыть ему все адаптеры сразу. Стандартный приём: итератор — отдельная структура, хранящая состояние обхода и ссылку на источник (или владеющая им). Здесь же полезно увидеть IntoIterator: трейт, позволяющий писать `for x in collection`. Задание-скелет — кольцевой буфер из урока 1.18: метод `iter()` возвращает RingIter, обходящий элементы по порядку без аллокаций.

```
struct Ring { buf: Vec<i32>, head: usize, len: usize }

struct RingIter<'a> { ring: &'a Ring, offset: usize }

impl Ring {
    fn iter(&self) -> RingIter<'_> { RingIter { ring: self, offset: 0 } }
}

impl<'a> Iterator for RingIter<'a> {
    type Item = i32;
    fn next(&mut self) -> Option<i32> {
        if self.offset >= self.ring.len { return None; }
        let idx = (self.ring.head + self.offset) % self.ring.buf.len();
        self.offset += 1;
        Some(self.ring.buf[idx])
```

```
    }
}

fn main() {
    letmut r = Ring { buf: vec![0; 4], head: 0, len: 0 };
    for v in [1, 2, 3, 4, 5] { r.buf[(r.head + r.len) % 4] = v; r.len = (r.len + 1).min(4); r.head =
(r.head + if r.len == 4 { 1 } else { 0 }) % 4; }
    let evens: Vec<i32> = r.iter().filter(|x| x % 2 == 0).collect();
    println!("{:?}", evens);   // итератор Ring работает со ВСЕМИ адаптерами
}
```

ОБРАТИТЕ ВНИМАНИЕ

Анонимный lifetime `'_` в возвращаемом типе — элиминация из урока 1.13: связь с &self компилятор выводит сам.

ДОМАШКА Чистая реализация: переоформить main-код кольца в аккуратный Ring::push, добавить IntoIterator для &Ring, DoubleEndedIterator (next_back) и тест на пять адаптеров (map/filter/zip/take/sum).

УРОК 2.9 30 мин

Box, Rc, RefCell: когда строгого владения мало

ЦЕЛЬ — знать три инструмента «выхода из тупика» и их цену, выбирать их осознанно.

Три кейса, где чистое владение не выражает структуру. (1) Рекурсивный тип (список, дерево): размер должен быть конечным — `Box<T>` разрывает цикл вычисления размера, храня данные в куче. (2) Много владельцев одного значения: `Rc<T>` — счётчик ссылок (только однопоточно; для потоков — `Arc`, модуль 4), аналог `shared_ptr`. (3) Внутренняя мутабельность при общих ссылках: `RefCell<T>` переносит правила заимствования из компиляции в рантайм — нарушение даёт панику, а не UB. Комбинация `Rc<RefCell<T>>` — «общая мутабельная ячейка»: мощно, но это явный признак, что данные лучше переорганизовать; в производственном коде такие конструкции должны быть редкостью и объясняться в комментарии.

```
use std::{cell::RefCell, rc::Rc};

// 1) Box: рекурсивный тип
enum Tree { Leaf(i32), Node(Box<Tree>, Box<Tree>) }
fn sum(t: &Tree) -> i32 {
    match t {
        Tree::Leaf(v) => *v,
        Tree::Node(l, r) => sum(l) + sum(r),
    }
}

// 2)+3) Rc + RefCell: общий редактируемый документ
fn main() {
    let t = Tree::Node(Box::new(Tree::Leaf(1)), Box::new(Tree::Node(
        Box::new(Tree::Leaf(2)), Box::new(Tree::Leaf(3)))));
    println!("sum = {}", sum(&t));

    let doc = Rc::new(RefCell::new(String::from("черновик")));
    let reader = Rc::clone(&doc);          // счётчик: 2
    doc.borrow_mut().push_str(" v2");
    println!("{reader:?} refs={}", Rc::strong_count(&doc));
}
```

СРАВНЕНИЕ

C++: `unique_ptr/shared_ptr` — то же самое, но RefCell без аналога: const-методы могут мутировать через mutable/const_cast. Go: всё просто можно — указатели и GC; цена — невозможность статически увидеть граф владения. Rc-циклы дают утечку точно так же, как shared_ptr-циклы: против них — `Weak`.

ДОМАШКА Реализовать двусвязный список игрушечного размера (3–4 метода) на `Rc<RefCell<Node>>` + Weak; задокументировать, почему prev — Weak. Честно оценить: приятно ли этим пользоваться, и что вы выбрали бы вместо него.

УРОК 2.10 25 мин

Синтез: переписываем «по-растовски»

ЦЕЛЬ — взять решение недели 4 и отрефакторить его на трейты, генерики и итераторы; почувствовать разницу стиля.

Берём читатель конфигов из урока 1.28 и переводим на уровень модуля 2: парсер становится генерик-функцией с трейтом `FromStr`-подобного контракта, ошибки — типизированные, вывод — итераторная цепочка, тесты — табличные. Это эталонное упражнение курса: разница «до/после» наглядно показывает, что идиоматичность — не украшение, а сжатие кода при том же поведении. Рефакторинг ведётся через зелёные тесты: поведение не меняется, сигнатуры — да.

```
use std::collections::HashMap;

trait ParseValue: Sized {
    fn parse_value(s: &str) -> Result<Self, String>;
}

impl ParseValue for u16 {
    fn parse_value(s: &str) -> Result<Self, String> {
        s.parse().map_err(|e| format!("u16: {e}"))
    }
}
impl ParseValue for bool {
    fn parse_value(s: &str) -> Result<Self, String> {
        match s { "true" => Ok(true), "false" => Ok(false), _ => Err("bool: true|false".into()) }
    }
}

fn get<T: ParseValue>(kv: &HashMap<String, String>, key: &str) -> Result<Option<T>, String> {
    kv.get(key).map(|v| T::parse_value(v)).transpose()
}

#[test]
fn table() {
    let kv: HashMap<_, _> = [("port", "9000"), ("verbose", "true")]
        .iter().map(|(k, v)| (k.to_string(), v.to_string())).collect();
    assert_eq!(get::<u16>(&kv, "port").unwrap(), Some(9000));
    assert_eq!(get::<bool>(&kv, "verbose").unwrap(), Some(true));
}
```

ЧЕК-ТОЧКА МОДУЛЯ

После этого занятия прогоните чек-лист модуля 2. Далее — модуль 3, где итераторы и генерики станут рабочими инструментами алгоритмики.

ДОМАШКА Дописать в rsgrep трейт `LineSource` (файл и stdin как источники строк) с генерик-обработкой; проверить, что main стал короче, а тесты — обобщённее.

КОНТРОЛЬ

Чек-лист модуля 2: умею / написал / объясню

- [ ] **Объясняю** разницу мономорфизации и dyn-диспетчеризации и выбираю осознанно.
- [ ] **Объясняю** Fn/FnMut/FnOnce через владение захваченных переменных.
- [ ] **Объясняю** зачем RefCell и чем опасен Rc-цикл.
- [ ] **Написал** свой итератор (next + IntoIterator) с тестами адаптеров.
- [ ] **Написал** трейт с ассоциированным типом и where-границами.
- [ ] **Умею** перечислить 8 ключевых трейтов std и назначение каждого.
- [ ] **Умею** заменить ручной цикл итераторной цепочкой без потери производительности.
- [ ] **Умею** вынести повторяющуюся логику в трейт с дефолтными методами.

МОДУЛЬ 3 · НЕДЕЛИ 7–8 · ЗАНЯТИЯ 3.1–3.10

Алгоритмы и структуры данных на Rust

2 недели · 10 занятий · ≈ 4 ч теории · лаба 2: algolib · criterion-бенчмарки

Цель модуля — довести алгоритмическую подготовку до уровня «пишу основные алгоритмы на Rust не переводя в голове с C++». Мы реализуем классику своими руками, а не только вызываем std: сортировки, бинарный поиск, стеки и очереди, деревья, кучи, обходы графов и динамическое программирование. Параллельно решаются инженерные вопросы, специфичные для Rust: как обобщать алгоритм по типу элемента через трейты Ord, как работать на срезах без лишних аллокаций, как строить графы на `Vec<Vec<usize>>` вместо ссылок и почему это чаще всего лучший вариант. Каждая реализация сопровождается бенчмарком через criterion — культура измерений вводится сразу, а не «когда-нибудь потом».

НЕДЕЛЯ 7

Сложность, сортировки, поиск

УРОК 3.1 25 мин

Сложность и измерение: criterion как рефлекс

ЦЕЛЬ — восстановить Big-O-рамку и поставить бенчмаркинг в ежедневный цикл.

Все знают O(n log n) наизусть, но практика начинается, когда сложность проверяется измерением, а не мнемоникой. Крейт `criterion` даёт статистически честные замеры: прогрев, выбросы, сравнение с прошлым прогоном. Правило курса: любой алгоритм из модуля снабжается бенчмарком на трёх объёмах (10^3, 10^5, 10^6 элементов) — кривая на графике сразу показывает, где кончается константа и начинается асимптотика. Бенчмарки живут в `benches/` и запускаются отдельной командой, чтобы не мешать тестам.

CARGO.TOML

```
[dev-dependencies]
```

```
criterion = "0.5"

[[bench]]
name = "sorting"
harness = false

<span class="cap">benches/sorting.rs</span>
use criterion::{black_box, criterion_group, criterion_main, Criterion};

fn bench_quick(c: &mutCriterion) {
    letmut group = c.benchmark_group("sort");
    for size in [1_000u32, 100_000, 1_000_000] {
        let data: Vec<u64> = (0..size).rev().collect();   // худший вход
        group.bench_with_input(format!("reverse-{size}"), &data, |b, d| {
            b.iter(|| { letmut v = d.clone(); v.sort_unstable(); black_box(v); })
        });
    }
    group.finish();
}
criterion_group!(benches, bench_quick);
criterion_main!(benches);
```

СРАВНЕНИЕ

Go: `go test -bench` — тот же принцип, менее богата статистика. C++: google-benchmark — ближайший родственник criterion. Общее железное правило: `black_box` обязателен, иначе оптимизатор выкинет весь замер.

ДОМАШКА Поставить criterion в algolib (каркас лабы 2); замерить `Vec::contains` против `HashSet::contains` на 10^5 поисках; сохранить вывод в docs/bench-01.md с выводом, где какой вариант уместен.

УРОК 3.2 30 МИН

Учебные сортировки: пузырёк, вставки, выбор — на срезах

ЦЕЛЬ — написать $O(n^2)$-сортировки на &mut [T] и привыкнуть к работе с индексами вместо итераторов при мутации.

Учебные сортировки полезны не алгоритмом (их трогать в прод не будут), а дисциплиной Rust: мутация среза по индексам, обмен элементов через `swap`, обобщение через `T: Ord`. Важный нюанс: `v[i] > v[j]` для генерика не скомпилируется — нужно `v[i] > v[j]` через дескрипторы Ord: доступ по индексу на срезе даёт `T` по значению только для Copy; для общего T сравниваем ссылки. Из этого растёт понимание, почему в std всё написано на индексах и `ptr`-арине.

```
pub fn insertion_sort<T: Ord>(a: &mut [T]) {
    for i in 1..a.len() {
        let mut j = i;
        while j > 0 && a[j - 1] > a[j] {   // a[j-1] > a[j] работает: Ord на T
            a.swap(j - 1, j);
            j -= 1;
        }
    }
}

pub fn selection_sort<T: Ord>(a: &mut [T]) {
    for i in 0..a.len() {
        let min = (i..a.len()).min_by(|&x, &y| a[x].cmp(&a[y])).unwrap();
        a.swap(i, min);
    }
}
```

```rust
#[test]fn both_match_std() {
    formut src invec![vec![5, 3, 1], vec![], vec![1], vec![2, 2, 2]] {
        letmut a = src.clone(); insertion_sort(&mut a);
        src.sort(); assert_eq!(a, src);
        letmut b = src.clone(); selection_sort(&mut b); assert_eq!(b, src);
    }
}
```

СРАВНЕНИЕ

C++: `std::swap(a[i], a[j])` — то же; отличие в требовании `T: Ord`, которое Rust диктует явно. Go: интерфейс sort.Interface — динамическая диспетчеризация каждого сравнения; Rust-генерик инлайнит компаратор.

ДОМАШКА В algolib: bubble_sort (с ранним выходом), insertion_sort, selection_sort + тест на 10^4 случайных значений против std::sort + criterion-бенчмарк всех трёх.

УРОК 3.3 30 мин

Merge sort: рекурсия на срезах и split_at_mut

ЦЕЛЬ — реализовать устойчивую O(n log n) сортировку и освоить безопасное деление мутабельного среза.

Merge sort — первая «настоящая» сортировка модуля: устойчивая (stable), предсказуемая O(n log n) на любом входе, идеально ложится на срезы. Ключевой Rust-инструмент — `split_at_mut`: делит срез на две независимые мутабельные половины там, где в C++ вы бы просто взяли два указателя. Рекурсия останавливается на малых длинах — там выгоднее вставки (гибрид — это уже Timsort, который в std). Слияние — классический двухпальцевый проход с `Vec::with_capacity` под итоговый размер.

```rust
pub fn merge_sort<T: Ord + Clone>(a: &mut [T]) {
    if a.len() ⩽ 32 { insertion_sort(a); return; }
    let mid = a.len() / 2;
    let (l, r) = a.split_at_mut(mid);   // два независимых &mut среза
    merge_sort(l);
    merge_sort(r);
    merge(a, mid);
}

fn merge<T: Ord + Clone>(a: &mut [T], mid: usize) {
    let (left, right) = (a[..mid].to_vec(), a[mid..].to_vec());
    let (mut i, mut j, mut k) = (0, 0, 0);
    while i < left.len() && j < right.len() {
        if left[i] ⩽ right[j] { a[k] = left[i].clone(); i += 1; }
        else                  { a[k] = right[j].clone(); j += 1; }
        k += 1;
    }
    while i < left.len()  { a[k] = left[i].clone();  i += 1; k += 1; }
    while j < right.len() { a[k] = right[j].clone(); j += 1; k += 1; }
}

#[test]
fn stability_and_correctness() {
    let mut v: Vec<(u8, u8)> = vec![(2, 1), (1, 9), (2, 3), (1, 4)];
    merge_sort(&mut v);
    assert_eq!(v, vec![(1, 9), (1, 4), (2, 1), (2, 3)]); // порядок пар сохранён
}
```

> ПРОИЗВОДИТЕЛЬНОСТЬ
> Clone в слиянии — плата за простоту. В версии «со звёздочкой» лабы: буфер, переиспользуемый между рекурсиями, убирает аллокации и ускоряет merge в 2–3 раза — убедитесь бенчмарком.

ДОМАШКА merge_sort в algolib + тесты (пустой, одинаковые, обратно-упорядоченный, 10^6 случайных) + бенчмарк против `sort_unstable`; в комментарии — почему std быстрее и за счёт чего.

УРОК 3.4 25 мин

Quicksort, Ord против PartialOrd, std-выбор

ЦЕЛЬ — реализовать быструю сортировку и разобраться, почему Ord и PartialOrd — разные трейты.

Quicksort на Rust отлично показывает роль инвариантов: partition по Ломуто с одним `swap` на обмен, рекурсия на поддиапазонах. Трейт `Ord` — тотальный порядок (для типов, где сравнение всегда определено), `PartialOrd` — частичный (f64 с NaN). Сортировать f64 напрямую нельзя — нет Ord; обёртка-компаратор через `total_cmp` или сортировка по ключу решают вопрос. В std: `sort` — устойчивый Timsort-подобный, `sort_unstable` — ipnsort/pdq-семейство, обычно быстрее; выбирать по требованию устойчивости, а не «на глаз».

```
pub fn quicksort<T: Ord>(a: &mut [T]) {
    if a.len() ⩽ 1 { return; }
    let pivot = partition(a);
    let (l, r) = a.split_at_mut(pivot);
    quicksort(l);
    quicksort(&mut r[1..]);
}

fn partition<T: Ord>(a: &mut [T]) → usize {
    let n = a.len();
    let mut i = 0;
    for j in 0..n - 1 {
        if a[j] ⩽ a[n - 1] { a.swap(i, j); i += 1; }
    }
    a.swap(i, n - 1);
    i
}

fn main() {
    let mut f = vec![1.5, f64::NAN, -0.0, 3.0];
    // f.sort();                                // ошибка: f64 не Ord
    f.sort_by(f64::total_cmp);                  // NaN аккуратно уходит в конец
    println!("{f:?}");
}
```

ДОМАШКА quicksort с медианой-из-трёх в algolib; бенчмарк на отсортированном входе (классический деградирующий случай для Ломуто) и обоснование выбора пивота; сравнить все четыре сортировки algolib одним criterion-графиком.

УРОК 3.5 25 мин

Бинарный поиск и два указателя

ЦЕЛЬ — освоить partition_point/бинарный поиск без ошибок на границах и шаблон «два указателя».

Бинарный поиск — вечный источник off-by-one. Rust-срез даёт готовый `partition_point` — позицию первого элемента, не удовлетворяющего предикату, — и это закрывает 90% задач: lower_bound, upper_bound, вставка с сохранением порядка. Ручную версию пишем один раз для понимания инвариантов: полузакрытый диапазон [lo, hi) и выход по lo == hi. Шаблон «два

указателя» решает пару-сумму в отсортированном массиве и задачи сжатия — здесь он приходит в итераторной форме.

```rust
pub fn lower_bound<T: Ord>(a: &[T], x: &T) -> usize {
    a.partition_point(|v| v < x)          // первый индекс с a[i] ≥ x
}

pub fn binary_search_manual<T: Ord>(a: &[T], x: &T) -> Option<usize> {
    let (mut lo, mut hi) = (0usize, a.len());
    while lo < hi {
        let mid = lo + (hi - lo) / 2;      // без переполнения
        match a[mid].cmp(x) {
            std::cmp::Ordering::Less    => lo = mid + 1,
            std::cmp::Ordering::Equal   => return Some(mid),
            std::cmp::Ordering::Greater => hi = mid,
        }
    }
    None
}

fn two_sum_sorted(a: &[i32], target: i32) -> Option<(usize, usize)> {
    let (mut i, mut j) = (0usize, a.len());
    // ЗАДАНИЕ: допишите двухпальцевый проход —
    // сумма a[i] + a[j-1]: двигаем i вправо при меньшей, j влево при большей.
    todo!("допишите проход за O(n)")
}
```

> **УЧЕБНАЯ ЗАГОТОВКА**
> Макрос `todo!()` компилируется, но паникует при вызове — так помечают недописанные ветки в реальных проектах. Тесты лабы не пройдут, пока проход не дописан.

ДОМАШКА В algolib: binary_search_manual, lower_bound, upper_bound, вставка в отсортированный Vec; two_sum_sorted двумя способами (два указателя и бинарный поиск) + тесты на дубликаты и отсутствие пары.

НЕДЕЛЯ 8 Стеки, деревья, кучи, графы, ДП

УРОК 3.6 25 мин

Стеки, очереди, деки: VecDeque и классические задачи

ЦЕЛЬ — решать стандартные задачи на стек/очередь идиоматично: VecDeque, matches!,pop-цепочки.

`Vec<T>` — готовый стек (push/pop с конца, O(1)); двусторонняя очередь — `VecDeque` (кольцевой буфер: push/push_back, pop/pop_front). Классика на стеке — валидные скобочные последовательности и «очередь из двух стеков»; на очереди — обход уровнями. Обратите внимание на `matches!` — макрос-свитч по шаблону, делающий ветвление однострочным, и на аккуратное pop: `while let Some(x) = stack.pop()`.

```rust
use std::collections::VecDeque;

pub fn brackets_ok(s: &str) -> bool {
    let mut st: Vec<char> = Vec::new();
    for c in s.chars() {
        match c {
            '(' | '[' | '{' => st.push(c),
            ')' | ']' | '}' => match (st.pop(), c) {
                (Some('('), ')') | (Some('['), ']') | (Some('{'), '}') => {}
                _ => return false,
            },
```

```
            _ => {}
        }
    }
    st.is_empty()
}

fn main() {
    let mut q = VecDeque::from([1, 2, 3]);
    q.push_back(4); q.push_front(0);
    println!("{q:?} front={:?} back={:?}", q.front(), q.back());
    println!("{} {} {}", brackets_ok("([{}])"), brackets_ok("([)"), brackets_ok(""));
}
```

ДОМАШКА Очередь на двух стеках (амортизированная O(1) pop) в algolib + тесты; задача «минимум в окне размера k» (монотонный дек) с бенчмарком против наивного окна.

УРОК 3.7 30 мин

Деревья на Box: BST и обходы

ЦЕЛЬ — построить BST с рекурсивными методами и увидеть, как владение выражает структуру дерева.

Дерево — первая рекурсивная структура данных на владении: `Box<Node>` делает размер конечным, а Option в поле — «нет ребёнка». Вставка по значению, поиск по ссылке, три обхода (in/pre/post). Рекурсивные методы принимают `&self` и `&mut self` соответственно; удаление узла с двумя детьми — классическая задача «минимум правого поддерева», здесь опущена до домашки со звёздочкой, чтобы не раздувать занятие. Отдельно проговариваем: в Rust BST на Box — простой и безопасный код; AVL/красно-чёрные деревья руками не пишем, для них есть `BTreeMap`.

```
#[derive(Debug)]
struct Node { val: i32, left: Option<Box<Node>>, right: Option<Box<Node>> }

#[derive(Debug, Default)]
struct Bst(Option<Box<Node>>);

impl Bst {
    fn insert(&mut self, v: i32) {
        match self.0 {
            None => self.0 = Some(Box::new(Node { val: v, left: None, right: None })),
            Some(ref mut n) => {
                if v < n.val { n.left.insert(v) }
                else if v > n.val { n.right.insert(v) }
            }
        }
    }
    fn contains(&self, v: i32) -> bool {
        match &self.0 {
            None => false,
            Some(n) => match v.cmp(&n.val) {
                std::cmp::Ordering::Equal => true,
                std::cmp::Ordering::Less => n.left.contains(v),
                std::cmp::Ordering::Greater => n.right.contains(v),
            },
        }
    }
    fn inorder(&self, out: &mut Vec<i32>) {
        if let Some(n) = &self.0 {
            n.left.inorder(out); out.push(n.val); n.right.inorder(out);
        }
    }
}

fn main() {
```

```
    letmut t = Bst::default();
    for v in [5, 3, 8, 1, 4] { t.insert(v); }
    letmut sorted = Vec::new(); t.inorder(&mut sorted);
    println!("{sorted:?} has4={} has7={}", t.contains(4), t.contains(7));
}
```

ДОМАШКА BST в algolib: insert/contains/min/height + обходы; тесты на вырожденный случай (вставка отсортированной последовательности — дерево превращается в список, height == n). Со звёздочкой: remove с двумя детьми.

УРОК 3.8 25 мин

Кучи и приоритетная очередь: BinaryHeap, top-K

ЦЕЛЬ — освоить BinaryHeap (max-heap) и паттерн top-K за O(n log k).

`BinaryHeap<T>` — max-куча на Vec; для min-heap — реверс-обёртка `Reverse` или обратный Ord. Приоритетная очередь — та же куча, если в ней лежит (приоритет, значение). Классика — top-K: держим кучу размера k, каждый новый элемент выталкивает худший; сложность O(n log k) против полной сортировки O(n log n). Реализуем свою min-heap на Vec с sift-up/sift-down — это проверка понимания инвариантов кучи, после чего std-версия становится прозрачной.

```
use std::{cmp::Reverse, collections::BinaryHeap};

fn top_k<T: Ord + Clone>(items: &[T], k: usize) -> Vec<T> {
    let mut h: BinaryHeap<Reverse<T>> = BinaryHeap::new();  // min-heap по Reverse
    for it in items {
        h.push(Reverse(it.clone()));
        if h.len() > k { h.pop(); }
    }
    h.into_iter().map(|Reverse(v)| v).collect()
}

#[test]
fn top_k_works() {
    let v = vec![5, 1, 9, 3, 7, 2];
    assert_eq!(top_k(&v, 3), vec![1, 2, 3]);
}

fn main() {
    let mut h = BinaryHeap::from(vec![3, 8, 1]);
    h.push(5);
    while let Some(x) = h.pop() { print!("{x} "); }   // 8 5 3 1
}
```

ДОМАШКА Своя MinHeap<T: Ord> на Vec: push/pop/peek + sift-методы, тесты и бенчмарк против BinaryHeap<Reverse> на 10^6 элементов; top-K для словаря частот из ДЗ 1.17.

УРОК 3.9 30 мин

Графы: представление, BFS и DFS

ЦЕЛЬ — строить графы на Vec<Vec<usize>> и писать BFS/DFS без ссылочной паутины.

Главный практический совет модуля: в Rust графы почти всегда лучше представлять индексами в `Vec`, чем узлами со ссылками `Rc<RefCell<Node>>`. Список смежности — `Vec<Vec<usize>>`: быстрый, кешируемый, без borrow-гимнастики. BFS — на VecDeque с массивом visited; DFS — рекурсией (простота) или явным стеком (большие графы без

переполнения стека). Диаметр/компоненты связности получаются обёрткой того же BFS. Этот же подход — стандарт в embedded (модуль 6-7), где ссылки и аллокации нежелательны.

```rust
use std::collections::VecDeque;

struct Graph { adj: Vec<Vec<usize>> }

impl Graph {
    fn new(n: usize) -> Self { Graph { adj: vec![Vec::new(); n] } }
    fn edge(&mut self, a: usize, b: usize) {
        self.adj[a].push(b); self.adj[b].push(a);   // неориентированный
    }
    fn bfs_levels(&self, start: usize) -> Vec<Option<u32>> {
        let mut dist = vec![None; self.adj.len()];
        let mut q = VecDeque::new();
        dist[start] = Some(0); q.push_back(start);
        while let Some(v) = q.pop_front() {
            for &to in &self.adj[v] {
                if dist[to].is_none() {
                    dist[to] = Some(dist[v].unwrap() + 1);
                    q.push_back(to);
                }
            }
        }
        dist
    }
    fn dfs_rec(&self, v: usize, seen: &mut Vec<bool>, out: &mut Vec<usize>) {
        if seen[v] { return; }
        seen[v] = true; out.push(v);
        for &to in &self.adj[v] { self.dfs_rec(to, seen, out); }
    }
}
```

> СРАВНЕНИЕ
> Go: тот же подход индексов (adj [][]int) — привычен; C++-привычка с Node* и указателями здесь специально «выключена»: индексное представление проще и быстрее.

ДОМАШКА Graph в algolib: bfs_levels, dfs_rec, dfs_iter (явный стек), компоненты связности; тесты на связный, несвязный и «звёздный» графы. Замер: 10^6 рёбер — время построения и BFS.

УРОК 3.10 30 мин

Динамическое программирование: мемоизация и таблицы

ЦЕЛЬ — писать ДП в двух стилях (мемоизация через HashMap, таблица на Vec) и выбирать между ними.

Динамическое программирование в Rust удобно в двух формах. Мемоизация рекурсии — HashMap<Key, Value> внутри структуры-решателя (самоприменение: замыкания не любят рекурсию, поэтому состояние выносится в struct). Табличная форма — Vec<T> с заполнением по индексам: быстрее, без хеширования, лучше по памяти; выбор — по размеру ключевого пространства. Разбираем рюкзак 0/1 и наибольшую возрастающую подпоследовательность (LIS) за O(n log n) через lower_bound из урока 3.5 — сразу видно, как модуль сцепляется в одну систему.

```rust
pub fn knapsack(weights: &[u32], values: &[u32], cap: u32) -> u32 {
    let cap = cap as usize;
    let mut dp = vec![0u32; cap + 1];                 // dp[w] — лучший вес ≤ w
    for (&w, &v) in weights.iter().zip(values) {
        for c in (w as usize..=cap).rev() {           // реверс = «взять один раз»
            dp[c] = dp[c].max(dp[c - w as usize] + v);
        }
```

```
    }
    dp[cap]
}

pubfn lis<T: Ord + Clone>(a: &[T]) → Vec<T> {
    letmut tails: Vec<T> = Vec::new();
    for x in a {
        let pos = tails.partition_point(|t| t < x);   // lower_boundif pos == tails.len() {
tails.push(x.clone()); } else { tails[pos] = x.clone(); }
    }
    tails                                             // длина = LIS (сами элементы — не всегда LIS)
}

#[test]fn classic() {
    assert_eq!(knapsack(&[2, 3, 4], &[3, 4, 5], 5), 7);
    assert_eq!(lis(&[3, 1, 8, 2, 9]).len(), 3);
}
```

ДОМАШКА + ЛАБА Сдать **лабу 2 (algolib)**: 4 сортировки, поиск, стек/очередь, BST, MinHeap, графы (BFS/DFS/компоненты), рюкзак + LIS; criterion-бенчмарки; README со сложностями; код-ревью по чек-листу. Итог недели: алгоритмическая библиотека, которую вы будете цитировать на собеседованиях.

КОНТРОЛЬ

Чек-лист модуля 3: умею / написал / объясню

- ☐ **Написал** 4 сортировки + lower_bound + binary search с тестами и бенчмарками.
- ☐ **Написал** BST, MinHeap, BFS/DFS на индексных графах — без Rc-паутины.
- ☐ **Написал** рюкзак и LIS; знаю, когда мемоизация, а когда таблица.
- ☐ **Умею** обобщать алгоритм через T: Ord и писать бенчмарк criterion.
- ☐ **Объясняю** выбор sort vs sort_unstable и Ord vs PartialOrd.
- ☐ **Объясняю**, почему граф на Vec<Vec<usize>> часто лучше ссылочной структуры.

МОДУЛЬ 4 · НЕДЕЛЯ 9 · ЗАНЯТИЯ 4.1–4.5

Конкурентность и асинхронность

1 неделя · 5 занятий · ≈ 2 ч теории · std::thread · каналы · Tokio · Send/Sync

Конкурентность — там, где Rust-гарантии окупаются буквально: data race невозможен в безопасном коде, и это проверяется не рантайм-детектором, а системой типов через маркерные трейты Send и Sync. Модуль короткий, но плотный: сначала нативные потоки и каналы (привычная модель для Go-разработчика), затем асинхронная модель — Future, стеклесс-корутины и рантайм Tokio, без которого не существует ни один веб-фреймворк модуля 5. Ключевая мысленная рамка: горутина — это стековая корутина, спрятанная в рантайм; Rust-задача — стеклесс-конечный автомат, который вы видите и компилируете явно.

НЕДЕЛЯ 9 **Потоки, каналы, async/await**

УРОК 4.1 25 мин

std::thread: spawn, move, join

ЦЕЛЬ — запускать потоки и передавать в них данные, не «стреляя себе в ногу» заёмом.

`thread::spawn` принимает замыкание и запускает его в новом потоке ОС (1:1, в отличие от M:N-горутин). Замыкание обязано владеть всем, что использует: отсюда `move` — и здесь видна связь недели 2: «can not borrow data from a thread that might not live long enough» — это lifetime-контроль, а не формальность. Результат возвращается через `JoinHandle::join()` или каналы. Для параллельной обработки коллекций сначала пробуйте `rayon` (крейт): `par_iter().map().collect()` — готовый fork-join без ручных потоков.

```
use std::thread;

fn main() {
    let data = vec![1, 2, 3, 4, 5];
    let handle = thread::spawn(move || {          // move: владение data в поток
        data.iter().map(|x| x * x).sum::<i32>()
    });
    println!("сумма квадратов = {}", handle.join().unwrap());

    let chunks: Vec<Vec<i32>> = (0..4).map(|i| vec![i; 100]).collect();
    let handles: Vec<_> = chunks.into_iter().map(|c| {
        thread::spawn(move || c.iter().sum::<i32>())
    }).collect();
    let total: i32 = handles.into_iter().map(|h| h.join().unwrap()).sum();
    println!("итого {total}");
}
```

СРАВНЕНИЕ

C++: `std::thread` + ручные мьютексы; передача по ссылке — UB, если поток пережил данные. Go: `go func()` дешевле, но захват переменной цикла — классический баг; Rust запрещает захват по ссылке в другой поток без Arc.

ДОМАШКА Параллельный подсчёт слов: разбить файл на N кусков, по потоку на кусок, свести HashMap'ы; сравнить время с однопоточным на 100 МБ текста. Затем — то же через rayon, три строки вместо тридцати.

УРОК 4.2 30 мин

Каналы и мьютексы: mpsc, Arc<Mutex<T>>

ЦЕЛЬ — освоить два способа обмена данными между потоками и правило «передавай сообщениями, шарь состояние редко».

`std::sync::mpsc` — канал множества производителей к одному потребителю: `send` передаёт владение значением (copy без clone невозможно — ещё одно следствие владения), receiver-конец можно итерировать до закрытия. Общее состояние — `Arc<Mutex<T>>`: атомарный счётчик ссылок + блокировка; доступ к данным возможен только через lock(), который даёт guard, отпускающий мьютекс при выходе из области — RAII в действии. Дедлоков

компилятор не ловит, но ловит гонки данных: распечатать из двух потоков одну HashMap без Mutex не скомпилируется вообще.

```
use std::sync::{mpsc, Arc, Mutex};
use std::thread;

fn main() {
    // канал: воркеры отправляют результаты
    let (tx, rx) = mpsc::channel();
    for id in 0..4 {
        let tx = tx.clone();
        thread::spawn(move || {
            tx.send(format!("результат воркера {id}")).unwrap();
        });
    }
    drop(tx);   // закрываем исходный конец: итерация закончится
    for msg in rx { println!("{msg}"); }

    // общее состояние: счётчик
    let counter = Arc::new(Mutex::new(0u64));
    let mut handles = Vec::new();
    for _ in 0..8 {
        let c = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            for _ in 0..1000 { *c.lock().unwrap() += 1; }
        }));
    }
    for h in handles { h.join().unwrap(); }
    println!("счётчик = {}", *counter.lock().unwrap());   // ровно 8000
}
```

> СРАВНЕНИЕ
>
> Go: `chan` — в языке; Rust-mpsc те же семантики, но компилятор не даёт отправить ссылку, только владение. `sync.Mutex` в Go не связан с данными — забыл lock, живи с гонкой; Rust связывает мьютекс с данными структурно: без lock() данные недостижимы.

ДОМАШКА Воркер-пул: N потоков берут задачи из общего `Arc<Mutex<VecDeque<Job>>>` + Condvar (или mpsc + shared receiver через Arc), обрабатывают, отвечают по каналу. Тест: 10^{00} задач, счётчик выполнения ровно N раз.

УРОК 4.3 30 мин

Send/Sync и async-модель: Future и executor

ЦЕЛЬ — понять маркерные трейты и устройство стеклесс-корутин до слова «Tokio».

`Send` — тип можно передать в другой поток; `Sync` — тип можно заимствовать (&T) из разных потоков. Это автотрейты, выводимые компилятором, — и именно они делают гонки данных невозможными: `Rc` — не Send, поэтому его не получится переслать в поток, компилятор остановит вас за строку до бага. Асинхронность: `async fn` возвращает `impl Future` — конечный автомат, который на каждом `await` сохраняет своё состояние; ничего не происходит, пока Future не опросит (poll) исполнитель (executor). Сравнение с Go: горутина — стековая корутина с переключением в рантайме, удобнее, но каждый воркер тянет стек; Rust-задача — объект в куче размером ровно под свои локальные переменные, это и позволяет ставить 100k задач на ESP32.

```
// Модель без рантайма: свой мини-исполнитель для понимания
use std::{future::Future, pin::Pin, task::{Context, Poll, RawWaker, Waker}};

struct Now(u64);
```

```
implFutureforNow {
    typeOutput = u64;
    fn poll(mutself: Pin<&mutSelf>, _cx: &mutContext<'_>) -> Poll<u64> {
        self.0 += 1;
        ifself.0 > 3 { Poll::Ready(42) } else { Poll::Pending }  // "ещё не готов"
    }
}

fn main() {
    // Полноценный блокирующий spin-исполнитель (учебный, не для прод):letmut fut = Now(0);
    let waker = std::task::Waker::noop();
    letmut cx = Context::from_waker(waker);
    let out = loop {
        match std::pin::pin!(fut.as_mut()).poll(&mut cx) { // poll до готовностиPoll::Ready(v) => break
v,
            Poll::Pending => std::thread::yield_now(),
        }
    };
    println!("future вернул {out}");
}
```

ВАЖНО ПРО ВЕРСИИ

API `Waker::noop()` — стабильно с недавних версий std; если ваша версия ругается, используйте крейт `futures::executor::block_on(fut)` — одна строка, тот же смысл. Идея занятия не в API, а в том, что Future — это просто конечный автомат с poll.

ДОМАШКА Текстом (10–15 предложений): почему Rc<T> не Send, а Arc<T> — Send; что случится с async-функцией, у которой между двумя await лежит долгая синхронная операция, и как это чинится spawn_blocking.

УРОК 4.4 25 мин

Tokio: runtime, задачи, select!, таймауты

ЦЕЛЬ — поставить Tokio и освоить ежедневный минимум: #[tokio::main], spawn, select!, time.

Tokio — фактический стандарт async-рантайма (мультипоточный work-stealing планировщик, таймеры, io). Ежедневный минимум: атрибут `#[tokio::main]` превращает main в runtime; `tokio::spawn` — «горутина»: возвращает JoinHandle задачи; `tokio::select!` — ожидание нескольких Future с победой первой готовой (отмена остальных); `tokio::time::{sleep, timeout}` — таймеры. Каналы Tokio (`mpsc::channel`) — асинхронные, с backpressure через ограниченную ёмкость. Это всё, что понадобится модулю 5: Axum сидит на Tokio напрямую.

CARGO.TOML

```
[dependencies]
tokio = { version = "1", features = ["full"] }

<span class="cap">src/main.rs</span>
use tokio::{sync::mpsc, time::{sleep, Duration, timeout}};

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel(16);          // ограниченный канал = backpressure

    let producer = tokio::spawn(async move {
        for i in 0..5 {
            sleep(Duration::from_millis(100)).await;   // «долгая» асинхронная работа
            tx.send(i).await.unwrap();
        }
```

```
    });

    whileletSome(v) = rx.recv().await {
        println!("получено {v}");
    }
    producer.await.unwrap();

    // гонка двух операций:let fast = sleep(Duration::from_millis(50));
    let slow = sleep(Duration::from_millis(500));
    tokio::select! {
        _ = fast ⇒ println!("быстрый успел"),
        _ = slow ⇒ println!("медленный успел"),
    }

    match timeout(Duration::from_millis(200), sleep(Duration::from_secs(1))).await {
        Err(_) ⇒ println!("таймаут сработал — отмена задачи"),
        Ok(_)  ⇒ unreachable!(),
    }
}
```

> СРАВНЕНИЕ
> Go: select — в языке, каналы — в языке; Tokio даёт то же поверх библиотеки. Ключевое отличие философии: Go прячет планировщик, Rust даёт выбрать/собрать рантайм (Tokio, smol, Embassy на МК).

ДОМАШКА Асинхронный «монитор»: 3 задачи-эмиттера (CPU, память, время) шлют события в общий mpsc; select! с таймаутом 1 с печатает тик, если событий нет; graceful shutdown через токен отмены (tokio_util::sync::CancellationToken).

УРОК 4.5 25 мин

Синтез: параллельный fetcher и граница sync/async

ЦЕЛЬ — собрать конкурентный инструмент и зафиксировать правила выбора между потоками и async.

Итоговое занятие — конкурентный загрузчик: список URL, ограничение параллелизма (семафор), таймауты, сбор результатов. Заодно — карта решений, которую вы будете применять весь остаток курса: CPU-bound работа → потоки (rayon); io-bound с тысячами соединений → async (Tokio); немного долгих io-операций → потоки проще; смешанный случай → async + `spawn_blocking` для CPU-участков. Крашевой камень модуля: осознать, что async-код синхронно блокировать нельзя (не отдадите рантайм поток), поэтому правило «не блокируй executor» входит в чек-листы всех последующих лаб.

```
use std::time::Duration;
use tokio::{sync::Semaphore, time::timeout};
use std::sync::Arc;

async fn fetch(url: &str, sem: Arc<Semaphore>) → (String, Result<usize, String>) {
    let _permit = sem.acquire().await.unwrap();     // ограничение параллелизма
    match timeout(Duration::from_secs(2), async {
        // в лабе: reqwest::get(url).await...
        Ok(url.len())                               // заглушка ради листинга
    }).await {
        Ok(Ok(n))  ⇒ (url.to_owned(), Ok(n)),
        Ok(Err(e)) ⇒ (url.to_owned(), Err(e)),
        Err(_)     ⇒ (url.to_owned(), Err("timeout".into())),
    }
}

#[tokio::main]
async fn main() {
    let urls = vec!["a.dev", "b.dev", "c.dev", "d.dev", "e.dev"];
```

```
letmut hs = Vec::new();
    for u in urls { hs.push(tokio::spawn(fetch(u, sem.clone()))); }
    for h in hs { let (u, r) = h.await.unwrap(); println!("{u}: {r:?}"); }
}
```

ДОМАШКА Довести fetcher: крейт reqwest (feature json), параллелизм 4, таймаут 2 с, ретрай один раз; вывод таблицей. Затем прогоните чек-лист модуля 4 — неделя закрыта.

КОНТРОЛЬ

Чек-лист модуля 4: умею / написал / объясню

- ☐ **Написал** параллельный wordcount на потоках и воркер-пул на каналах.
- ☐ **Написал** async-монитор и fetcher на Tokio с семафором и таймаутами.
- ☐ **Объясняю** Send/Sync и почему data race не компилируется.
- ☐ **Объясняю** разницу стековых (goroutine) и стеклесс (Future) корутин.
- ☐ **Умею** выбирать: потоки vs async vs spawn_blocking — с обоснованием.

МОДУЛЬ 5 · НЕДЕЛИ 10–12 · ЗАНЯТИЯ 5.1–5.15

CRUD-приложение: от обзора фреймворков до Axum + SQLx

3 недели · 15 занятий · ≈ 6 ч теории · лаба 3: REST API · PostgreSQL · JWT

Модуль 5 превращает языковую базу в работающий сервис. Неделя 10 намеренно начитана шире, чем требует итоговый стек: сначала короткое знакомство с тремя главными фреймворками — Actix-web, Rocket и Axum — каждый с рабочим мини-API, затем осознанный выбор. Основной трек модуля — Axum + SQLx + PostgreSQL: самый массовый продакшен-стек Rust 2026 года, построенный прямо на Tokio из модуля 4. Важно понимать: навыки переносимы — слои, DTO, валидация и транзакции одинаковы для любого фреймворка, меняется только синтаксис роутинга.

НЕДЕЛЯ 10

Ландшафт web-фреймворков и выбор стека

УРОК 5.1 · 20 мин

HTTP-стек Rust: как устроены слои

ЦЕЛЬ — увидеть слоистую архитектуру: hyper → tower → фреймворк, и перестать путать роли.

Под всеми фреймворками лежит `hyper` — HTTP-реализация поверх Tokio, аналог net/http в Go. Над ним `tower` — экосистема сервисов и middleware (таймауты, лимиты, трейсинг) в виде универсальных обёрток. Фреймворки — это удобные надстройки с роутингом и экстракторами: `axum` (от авторов Tokio, Tower-native), `actix-web` (самый старый и быстрый, свой actor-рантайм на Actix), `rocket` (декларативный, магия атрибутов, исторически «rails-

подобный»). Понимание слоёв даёт спокойствие: REST-логика не привязана к фреймворку, переезд — это слой роутера, не больше.

> СРАВНЕНИЕ
> Go: net/http + chi/gin — один слой. Rust: hyper (транспорт) + tower (middleware-композиция) + фреймворк (эргономика). Итоговый стек модуля: axum, потому что Tower-мидлвари переиспользуются и он вырос из самого Tokio.

ДОМАШКА Прочитать README трёх фреймворков (docs.rs/axum, actix.rs, rocket.rs); выписать: модель роутинга, как извлекаются данные запроса, как выглядит middleware — таблицей на 9 ячеек.

УРОК 5.2 30 мин

Мини-API на Actix-web

ЦЕЛЬ — за одно занятие получить работающий JSON-API на Actix-web и почувствовать его стиль.

Actix-web — ветеран производительности: собственный executor, макросный роутинг `#[post("/tasks")]`, экстракторы аргументов (`Path`, `Json`, `Query`), `web::Data` для общего состояния. Ниже — полный CRUD в памяти (без БД): видно всё — атрибуты, экстракторы, сериализацию serde. Запуск: `cargo run`, проверка curl'ом. Заметьте: бизнес-логика (функции над Vec<Task>) не зависит от фреймворка — это принцип всех трёх недель.

CARGO.TOML

```
[dependencies]
actix-web = "4"
serde = { version = "1", features = ["derive"] }
serde_json = "1"

<span class="cap">src/main.rs</span>
use actix_web::{get, post, web, App, HttpResponse, HttpServer, Responder};
use serde::{Deserialize, Serialize};
use std::sync::Mutex;

#[derive(Serialize, Deserialize, Clone)]
struct Task { id: u64, title: String, done: bool }

#[derive(Default)]
struct Store(Mutex<Vec<Task>>);

#[post("/tasks")]
async fn create(store: web::Data<Store>, body: web::Json<Task>) -> impl Responder {
    let mut v = store.0.lock().unwrap();
    v.push(body.into_inner());
    HttpResponse::Created().json(v.last().unwrap())
}

#[get("/tasks")]
async fn list(store: web::Data<Store>) -> impl Responder {
    HttpResponse::Ok().json(store.0.lock().unwrap().clone())
}

#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| App::new()
        .app_data(web::Data::new(Store::default()))
        .service(create).service(list))
```

```
        .bind(("127.0.0.1", 8080))?.run().await
}
```

ДОМАШКА Дописать GET /tasks/{id} (404 через Option) и DELETE в Actix-версии; curl-скрипт проверки всех четырёх операций.

УРОК 5.3 30 МИН

Мини-API на Rocket

ЦЕЛЬ — то же приложение на Rocket; увидеть разницу философии: максимум магии атрибутов.

Rocket исторически ставит эргономику выше всего: роуты — аннотации `#[get("/tasks/<id>")]`, возврат `Option<Json<T>>` автоматически превращается в 404, guard'ы и fairings закрывают аутентификацию и метрики. С 0.5 Rocket живёт на async/Tokio, так что производительность перестала быть аргументом против. Из коробкиRocket даёт типизированные формы, cookies и даже встроенный тестовый клиент — его `rocket::local` позволяет писать API-тесты без запуска сокета, что приятно удивит Go-разработчика.

CARGO.TOML

```
[dependencies]
rocket = { version = "0.5", features = ["json"] }
serde = { version = "1", features = ["derive"] }

<span class="cap">src/main.rs</span>
use rocket::{get, post, launch, routes, serde::json::Json, State};
use serde::{Deserialize, Serialize};
use std::sync::Mutex;

#[derive(Serialize, Deserialize, Clone)]
#[serde(crate = "rocket::serde")]
struct Task { id: u64, title: String, done: bool }

#[derive(Default)]
struct Store(Mutex<Vec<Task>>);

#[post("/tasks", format = "json", data = "<task>")]
fn create(store: &State<Store>, task: Json<Task>) -> Json<Task> {
    store.0.lock().unwrap().push(task.into_inner());
    Json(store.0.lock().unwrap().last().unwrap().clone())
}

#[get("/tasks/<id>")]
fn get_one(store: &State<Store>, id: u64) -> Option<Json<Task>> {   // None = 404
    store.0.lock().unwrap().iter().find(|t| t.id == id).cloned().map(Json)
}

#[launch]
fn rocket() -> _ {
    rocket::build().manage(Store::default()).mount("/", routes![create, get_one])
}
```

СРАВНЕНИЕ

Rocket ближе всего к «rails-ощущению»: меньше явных типов, больше магии. Actix — максимум контроля и производительности. Axum — компромисс: типизация + Tower-экосистема. Все три — на Tokio (Rocket 0.5+, Actix — на своём, но совместим).

ДОМАШКА Rocket-версия: дописать DELETE и вернуть 404 с JSON-ошибкой; написать тест через rocket::local::blocking::Client — без сети, быстро.

УРОК 5.4 30 МИН

Мини-API на Axum: основной трек

ЦЕЛЬ — воспроизвести тот же CRUD на Axum и освоить его базовые механизмы: Router, extractors, State.

Axum строит API из функций-хендлеров с экстракторами-аргументами: `Path`, `Query`, `Json`, `State`. Роутер — конструктор `Router::new().route("/tasks", get(list).post(create))`; общее состояние — `State<T>` из `with_state`, где T обычно `Arc<AppCtx>`. Ошибки хендлеров — любой тип, реализующий `IntoResponse`; здесь наш `AppError`-enum из духа урока 1.24 станет боевым. Axum — самый «растовский» из трёх: ошибки типов ведут к 500 на компиляции, а не к 500 в проде.

CARGO.TOML

```
[dependencies]
axum = "0.8"
tokio = { version = "1", features = ["full"] }
serde = { version = "1", features = ["derive"] }

<span class="cap">src/main.rs</span>
use axum::{extract::State, http::StatusCode, routing::get, Json, Router};
use serde::{Deserialize, Serialize};
use std::sync::{Arc, Mutex};

#[derive(Serialize, Deserialize, Clone)]
struct Task { id: u64, title: String, done: bool }

#[derive(Default)]
struct Store(Mutex<Vec<Task>>);

type Ctx = Arc<Store>;

async fn list(State(s): State<Ctx>) -> Json<Vec<Task>> {
    Json(s.0.lock().unwrap().clone())
}

async fn create(State(s): State<Ctx>, Json(mut t): Json<Task>) -> (StatusCode, Json<Task>) {
    t.id = s.0.lock().unwrap().len() as u64 + 1;
    s.0.lock().unwrap().push(t.clone());
    (StatusCode::CREATED, Json(t))
}

#[tokio::main]
async fn main() {
    let app = Router::new()
        .route("/tasks", get(list).post(create))
        .with_state(Arc::new(Store::default()));
    let listener = tokio::net::TcpListener::bind("127.0.0.1:8080").await.unwrap();
    axum::serve(listener, app).await.unwrap();
}
```

ДОМАШКА Axum-версия: GET /tasks/{id} с 404, DELETE /tasks/{id}, PATCH с частичным обновлением; сравнить сигнатуры хендлеров трёх фреймворков в таблице.

УРОК 5.5 25 мин

Сравнение и выбор: как переносить навыки между фреймворками

ЦЕЛЬ — осознанно выбрать стек для лабы 3 и зафиксировать карту соответствий «фреймворк ↔ концепция».

После трёх реализаций одного и того же API выбор перестаёт быть верой. Критерии: (1) размер сообщества и вакансий — у Axum/Actix он максимальный; (2) экосистема middleware — Tower у Axum; (3) стиль ошибок — type-safe экстракторы Axum; (4) документация и версия-стабильность. Наш выбор для основного трека — **Axum + SQLx**, но решение студента может отличаться при обосновании. Карта соответствий одинакова для всех: роут = маршрут; экстрактор = биндинг аргументов; State = DI-контейнер; middleware = перехватчик; IntoResponse = сериализация ответа. Значит, навык «веб-слой» у вас уже переносим.

Критерий	Actix-web 4	Rocket 0.5	Axum 0.8
Модель	Свой executor, макросы-роуты	Tokio, магия атрибутов	Tokio + Tower-native
Стиль	Контроль и скорость	Декларативность, «rails»	Типизация, экосистема
Middleware	Actix-middleware + wrap	Fairings	tower-http + свои слои
Кому подходит	Высокие нагрузки, прокси	Быстрые прототипы, MVP	Продакшен-сервисы, команды

РЕШЕНИЕ МОДУЛЯ

Лаба 3 пишется на Axum + SQLx + PostgreSQL. Если студент выбирает Actix/Rocket — переносится без изменения структуры слоёв; критерии приёмки лабы одинаковы.

ДОМАШКА Сдать «один API — два фреймворка»: репозиторий с двумя ветками (axum/ и actix/ или rocket/), общий README-сравнение; в PR — выводы о том, что перенеслось, а что переписалось.

НЕДЕЛЯ 11 SQLx: схема БД, запросы, транзакции

УРОК 5.6 25 мин

PostgreSQL: установка, схема проекта, миграции

ЦЕЛЬ — поднять БД, спроектировать схему задачи и внедрить SQLx-миграции в репозиторий.

Проект лабы 3 — «менеджер задач» с пользователями: таблицы `users` и `tasks`. PostgreSQL поднимается в Docker; схема хранится в миграциях SQLx — каталог `migrations/` с парными файлами up/down, применяемых командой `sqlx migrate run`. Миграции — часть истории git: разворачивание базы на новой машине — две команды, а не «какая у нас сейчас схема».

Отдельно проговариваем типы: `BIGSERIAL PRIMARY KEY`, `TIMESTAMPTZ` (всегда с зоной), `NOT NULL`-дисциплина и внешние ключи с `ON DELETE CASCADE`.

DOCKER-COMPOSE.YML (МИНИМУМ)

```
services:
  db:
    image: postgres:16-alpine
    environment: [POSTGRES_USER=app, POSTGRES_PASSWORD=app, POSTGRES_DB=tasks]
    ports: ["5432:5432"]

<span class="cap">migrations/0001_init.sql</span>
CREATE TABLE users (
    id           BIGSERIAL PRIMARY KEY,
    email        TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE tasks (
    id         BIGSERIAL PRIMARY KEY,
    user_id    BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title      TEXT NOT NULL,
    done       BOOLEAN NOT NULL DEFAULT false,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX tasks_user_idx ON tasks(user_id);

<span class="cap">терминал</span>
cargo install sqlx-cli
export DATABASE_URL=postgres://app:app@localhost/tasks
sqlx migrate run
```

ДОМАШКА Поднять compose-стек, накатить миграцию, создать обратную (drop-версию), откатить и применить снова; зафиксировать структуру миграций в README.

УРОК 5.7 30 мин

SQLx: подключение и compile-time проверка запросов

ЦЕЛЬ — подключить пул соединений и понять главную фишку SQLx — проверку SQL на этапе компиляции.

`sqlx::PgPool` — пул соединений, живущий в State приложения. Макросы `query!` и `query_as!` проверяют SQL и типы колонок во время компиляции против реальной схемы (нужен доступ к БД при сборке или кеш-файл .sqlx) — это «ORM без ORM»: никаких магических моделей, чистый SQL с проверкой типов. Результат маппится в структуру по именам колонок; ошибки — типизированные (`sqlx::Error::RowNotFound`). Для динамических запросов есть некешируемая форма `sqlx::query_as::<_, T>()`.

CARGO.TOML

```
[dependencies]
sqlx = { version = "0.8", features = ["runtime-tokio", "postgres", "chrono", "uuid", "migrate"] }

<span class="cap">src/model.rs</span>
use serde::Serialize;
use chrono::{DateTime, Utc};

#[derive(Serialize)]
pub struct Task {
    pub id: i64,
```

```
    pub user_id: i64,
    pub title: String,
    pub done: bool,
    pub created_at: DateTime<Utc>,
}

<span class="cap">src/repo.rs</span>
use sqlx::PgPool;

pubasyncfn list(pool: &PgPool, user_id: i64) -> Result<Vec<Task>, sqlx::Error> {
    sqlx::query_as!(Task,
        "SELECT id, user_id, title, done, created_at
         FROM tasks WHERE user_id = $1ORDERBY id DESC",
        user_id
    ).fetch_all(pool).await
}

pubasyncfn create(pool: &PgPool, user_id: i64, title: &str) -> Result<Task, sqlx::Error> {
    sqlx::query_as!(Task,
        "INSERTINTO tasks (user_id, title) VALUES ($1, $2)
         RETURNING id, user_id, title, done, created_at",
        user_id, title
    ).fetch_one(pool).await
}
```

СРАВНЕНИЕ

Go: sqlx/database/sql — строки с плейсхолдерами без проверки; ошибка в SQL видна в рантайме. Rust sqlx: опечатка в имени колонки — ошибка компиляции. Это то же отношение, что между go vet и полной type-safety.

ДОМАШКА Репозиторий-слой лабы: get_by_id, update (частичный через COALESCE), delete; compile-time проверку включить через .env + DATABASE_URL; умышленно сломать одну колонку и приложить скриншот ошибки компиляции в PR.

УРОК 5.8 25 мин

Запросы посложнее: пагинация, фильтры, LIKE и N+1

ЦЕЛЬ — писать листинговые эндпоинты как в проде: лимиты, курсорная пагинация, безопасные фильтры.

Списки всегда ограничиваются: LIMIT/OFFSET для простых случаев, курсорная пагинация (WHERE id < $cursor ORDER BY id DESC LIMIT $n) для больших таблиц. Фильтрация по опциональным параметрам — через `COALESCE($1, title) = title` или динамическую сборку query-builder'ом. Задача N+1 решается одним JOIN'ом или IN-запросом. Инъекции предотвращаются параметрами $1/$2 — конкатенация строк в SQL не допускается никогда; compile-time макросы делают это правило естественным.

```
#[derive(serde::Serialize, sqlx::FromRow)]
pub struct TaskRow {
    pub id: i64, pub title: String, pub done: bool,
    pub created_at: chrono::DateTime<chrono::Utc>,
}

pub struct ListQuery { pub done: Option<bool>, pub q: Option<String>, pub limit: i64, pub cursor: i64 }

pub async fn list_filtered(pool: &sqlx::PgPool, q: &ListQuery) -> Result<Vec<TaskRow>, sqlx::Error> {
    let pattern = format!("%{}%", q.q.as_deref().unwrap_or(""));
    sqlx::query_as::<_, TaskRow>(
        "SELECT id, title, done, created_at
         FROM tasks
```

```
AND ($1::boolean ISNULLOR done = $1)
          AND ($2::text = '' OR title ILIKE $3)
        ORDERBY id DESCLIMIT $5"
    )
    .bind(q.done).bind(q.q.clone().unwrap_or_default()).bind(pattern)
    .bind(q.cursor).bind(q.limit.min(100))       // серверный потолок лимита
    .fetch_all(pool).await
}
```

ПРОИЗВОДИТЕЛЬНОСТЬ

OFFSET на глубоких страницах — это полный скан до смещения. Для API лабы достаточно курсора по id; пометьте в README, где вы бы поставили индекс `(user_id, id DESC)`.

ДОМАШКА Эндпоинт GET /tasks с query-параметрами (done, q, limit, cursor) через axum::extract::Query; тесты: фильтр по done, поиск по подстроке, курсорная навигация до конца списка.

УРОК 5.9 25 мин

Транзакции: атомарные операции с БД

ЦЕЛЬ — освоить begin/commit/rollback и типовой кейс «создать юзера и первую задачу атомарно».

Транзакция в SQLx — это `pool.begin()`, дающий `Transaction`-объект: операции идут через него, commit — `tx.commit()`, дроп без commit — автоматический rollback (RAII, привет неделя 2). Внутри транзакции сигнатуры функций принимают `&mut Transaction<Postgres>` или обобщённый executor: `impl Executor<'_, Database = Postgres>`, чтобы работать и с пулом, и с транзакцией — это генерики в боевой роли. Дедлоки и уровни изоляции упоминаем, но глубоко — тема курса по БД, не этого.

```
use sqlx::{PgPool, Postgres, Transaction};

pub async fn register_with_task(
    pool: &PgPool, email: &str, hash: &str, first_task: &str,
) -> Result<(i64, i64), sqlx::Error> {
    let mut tx = pool.begin().await?;
    let user: (i64,) = sqlx::query_as(
        "INSERT INTO users (email, password_hash) VALUES ($1, $2) RETURNING id",
    ).bind(email).bind(hash).fetch_one(&mut *tx).await?;
    let task: (i64,) = sqlx::query_as(
        "INSERT INTO tasks (user_id, title) VALUES ($1, $2) RETURNING id",
    ).bind(user.0).bind(first_task).fetch_one(&mut *tx).await?;
    tx.commit().await?;                    // без commit — откат при drop
    Ok((user.0, task.0))
}
```

ДОМАШКА Тест атомарности: вызвать register_with_task с email, нарушающим UNIQUE на втором запуске, и убедиться, что первая задача тоже не записалась (rollback); приложить вывод EXPLAIN не нужно — достаточно двух прогонов и проверки таблицы.

УРОК 5.10 20 мин

Лаба 3 · старт: слои сервиса и репозитория

ЦЕЛЬ — оформить структуру проекта лабы 3 и зафиксировать границы слоёв.

Структура лабы: `main.rs` (сборка Router, конфиг, запуск), `model.rs` (структуры данных), `repo.rs` (только SQL), `service.rs` (бизнес-правила, транзакции), `api.rs` (хендлеры Axum, DTO, валидация), `error.rs` (AppError → HTTP). Правило: хендлер не содержит SQL,

репозиторий не знает про HTTP. Такое разделение уже окупилось в Go/C++-проектах — здесь оно же, только компилятор следит, чтобы repo не вернул 404, а api не увидел sqlx::Error напрямую.

SRC/ERROR.RS — ЕДИНАЯ ОШИБКА API

```rust
use axum::{http::StatusCode, response::{IntoResponse, Response}, Json};
use serde_json::json;

pub enum AppError {
    NotFound(String),
    BadRequest(String),
    Unauthorized,
    Internal(String),
}

impl IntoResponse for AppError {
    fn into_response(self) -> Response {
        let (status, msg) = match self {
            AppError::NotFound(m)   => (StatusCode::NOT_FOUND, m),
            AppError::BadRequest(m) => (StatusCode::BAD_REQUEST, m),
            AppError::Unauthorized  => (StatusCode::UNAUTHORIZED, "нет доступа".into()),
            AppError::Internal(m)   => (StatusCode::INTERNAL_SERVER_ERROR, m),
        };
        (status, Json(json!({ "error": msg }))).into_response()
    }
}

impl From<sqlx::Error> for AppError {           // "?" в хендлерах: sqlx → AppError
    fn from(e: sqlx::Error) -> Self {
        match e {
            sqlx::Error::RowNotFound => AppError::NotFound("запись не найдена".into()),
            _ => AppError::Internal(e.to_string()),
        }
    }
}
```

ЧЕК-ТОЧКА НЕДЕЛИ

Слои выделены, AppError подключён через «?», healthcheck-эндпоинт /healthz отвечает на пинге пула. Коммит и PR «lab3: skeleton».

ДОМАШКА Перенести все функции repo в сервисный слой (сигнатуры с AppError), хендлеры оставляем тонкими; тест репозитория против поднятой БД (sqlx::test).

НЕДЕЛЯ 12 Полный REST API: DTO, JWT, тесты, сдача

УРОК 5.11 25 мин

serde: DTO, теги и валидация входа

ЦЕЛЬ — отделить API-структуры от моделей БД и валидировать вход до бизнес-логики.

Одна структура на всё — мина замедленного действия: поля БД утекают в API (например, password_hash сериализуется в ответ). Поэтому DTO: `CreateTask` (вход), `TaskOut` (выход), `Task` (модель БД). serde даёт контроль: `#[serde(skip_serializing)]`, переименования, `deny_unknown_fields`. Валидация — крейт `validator` с derive: правила на полях, единая точка

ошибки 422. Принцип «границы валидны» упрощает всё внутри: сервис получает только корректные данные.

```rust
use serde::{Deserialize, Serialize};
use validator::Validate;

#[derive(Deserialize, Validate)]
pub struct CreateTask {
    #[validate(length(min = 1, max = 200, message = "заголовок: 1-200 символов"))]
    pub title: String,
    #[serde(default)]
    pub priority: u8,
}

#[derive(Serialize)]
pub struct TaskOut {
    pub id: i64,
    pub title: String,
    pub done: bool,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub due: Option<chrono::DateTime<chrono::Utc>>,
}

// хендлер: экстрактор Json<CreateTask> + .validate()? перед сервисом
```

ДОМАШКА Внедрить DTO в лабу: TaskOut без полей БД-внутренностей; валидатор на title/priority; тесты: пустой title → 422, лишние поля → 400 (deny_unknown_fields).

УРОК 5.12 30 мин

Аутентификация: JWT, argon2, extractor-мидлварь

ЦЕЛЬ — добавить регистрацию/логин и защиту эндпоинтов через кастомный extractor.

Классическая схема JWT: `POST /auth/register` (пароль → argon2-хеш), `POST /auth/login` (проверка, выдача токена с exp и sub), дальше — заголовок Authorization: Bearer. В Axum защита делается не «мидлварь на всё», а кастомный extractor `AuthUser`: хендлер принимает его аргументом, и только с ним эндпоинт защищён — тип в сигнатуре и есть контроль доступа. Токены подписываются HS256 (секрет в env); в проде рассмотрели бы RS256 и refresh-токены — зафиксируем как «за пределами лабы».

SRC/AUTH.RS

```rust
use axum::{async_trait, extract::FromRequestParts, http::request::Parts};

pub struct AuthUser { pub user_id: i64 }

#[async_trait]
impl FromRequestParts<AppCtx> for AuthUser {
    type Rejection = AppError;
    async fn from_request_parts(parts: &mut Parts, ctx: &AppCtx) -> Result<Self, AppError> {
        let token = parts.headers.get("authorization")
            .and_then(|v| v.to_str().ok())
            .and_then(|v| v.strip_prefix("Bearer "))
            .ok_or(AppError::Unauthorized)?;
        let data = jsonwebtoken::decode::<Claims>(token, &ctx.keys, &Default::default())
            .map_err(|_| AppError::Unauthorized)?;
        Ok(AuthUser { user_id: data.claims.sub })
    }
}

// защищённый эндпоинт = просто принимает AuthUser:
```

```
async fn my_tasks(State(ctx): State<AppCtx>, who: AuthUser) -> Result<Json<Vec<TaskOut>>, AppError> {
    Ok(Json(repo::list(&ctx.pool, who.user_id).await?))
}
```

> СРАВНЕНИЕ
>
> Go: middleware с контекстом (c.Get("user")), забыл проверить — эндпоинт открыт. Rust: не принял AuthUser — не увидел user_id; компилятор подскажет на code review даже без человека.

ДОМАШКА register/login/refresh-эндпоинты; все CRUD-маршруты задач — за AuthUser; тест: без токена 401, с просроченным 401, с валидным 200.

УРОК 5.13 25 мин

Наблюдаемость: tracing, конфигурация, graceful shutdown

ЦЕЛЬ — подключить структурные логи, конфиг из окружения и корректное завершение.

Крейт `tracing` — структурное логирование с контекстом (span'ы на запрос: method, path, latency) и `tracing-subscriber` с JSON-выгрузкой; tower-http TraceLayer добавляет логи ко всем запросам одной строкой. Конфигурация — из переменных окружения через `dotenvy` и типизированный `Config::from_env()` с валидацией на старте: упавшие на первом запуске настройки лучше ночной отладки. Graceful shutdown: axum::serve(...).with_graceful_shutdown(сигнал) — в полёте запросы завершаются, новые отклоняются; для Pi-демона (модуль 6) это же умение понадобится в systemd.

SRC/MAIN.RS (СКЕЛЕТ ЗАПУСКА)

```
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    dotenvy::dotenv().ok();
    tracing_subscriber::fmt()
        .with_env_filter(tracing_subscriber::EnvFilter::from_default_env())
        .json().init();

    let cfg = Config::from_env()?;
    let pool = sqlx::postgres::PgPoolOptions::new()
        .max_connections(cfg.db_max_conn)
        .connect(&cfg.database_url).await?;
    sqlx::migrate!().run(&pool).await?;

    let app = api::router(Arc::new(AppCtx::new(pool, cfg.clone())))
        .layer(tower_http::trace::TraceLayer::new_for_http());

    let listener = tokio::net::TcpListener::bind((cfg.host.as_str(), cfg.port)).await?;
    axum::serve(listener, app)
        .with_graceful_shutdown(shutdown_signal())
        .await?;
    Ok(())
}

async fn shutdown_signal() {
    tokio::signal::ctrl_c().await.expect("ctrl-c handler");
    tracing::info!("получен сигнал, завершаемся");
}
```

ДОМАШКА Конфиг (host, port, database_url, jwt_secret, log_level) из env; RUST_LOG=info; проверка graceful: curl с долгим запросом + Ctrl-C — сервер завершит ответ, потом выйдет.

УРОК 5.14 25 мин

Тестирование API и сдача лабы 3

ЦЕЛЬ — интеграционные тесты на живой БД и формальная приёмка лабы.

SQLx даёт атрибут `#[sqlx::test]`: тест получает изолированную БД с накатанными миграциями — интеграционные тесты становятся такими же рутинными, как unit. Полный цикл проверяем через oneshot-клиент Axum (Router без сети) или HTTP-клиентом на тестовом сервере. Критерии сдачи лабы 3: register → login → create → list → update → delete; 401/404/422-пути покрыты; docker-compose поднимает всё с нуля одной командой; README с curl-примерами; clippy чист.

TESTS/API.RS

```
use serde_json::json;

#[sqlx::test(migrations = "./migrations")]
async fn full_crud_flow(pool: sqlx::PgPool) {
    let app = api::router_for_test(pool).await;

    let reg = app.clone().oneshot(
        Request::builder().method("POST").uri("/auth/register")
            .header("content-type", "application/json")
            .body(Body::from(json!({"email": "a@b.io", "password": "s3cret"}).to_string())).unwrap()
    ).await.unwrap();
    assert_eq!(reg.status(), 201);

    // login → token; create с Bearer; list == 1; delete; list == 0
    // (полный код — в репозитории лабы)
}
```

ЛАБОРАТОРНАЯ РАБОТА №3 · 8% ИТОГОВОЙ ОЦЕНКИ

REST API «менеджер задач»: Axum + SQLx + PostgreSQL

- **Минимум:** auth (register/login, argon2+JWT), CRUD задач по пользователю, DTO + валидация, миграции, docker-compose, интеграционные тесты, README.
- **Продвинутый:** курсорная пагинация, фильтры/поиск, limit-guard, структурные логи, graceful shutdown.
- **Со звёздочкой:** refresh-токены, rate-limit через tower, CI на GitHub Actions (build + test + clippy).

ДОМАШКА Устранить замечания ревью лабы 3; выписать 5 вещей, которые Axum/SQLx сделали за вас (типизация запросов, экстракторы…) и 5, за которые всё ещё отвечаете вы.

УРОК 5.15 20 мин

Синтез модуля: от API к IoT-платформе

ЦЕЛЬ — сформулировать мост к модулям 6–8: API лабы 3 станет сервером проекта, узлы подключатся к нему.

Завершаем модуль взглядом вперёд: сервис задач лабы 3 получит два новых клиента. Raspberry Pi (модуль 6) будет отправлять показания датчика в `POST /metrics`; ESP32 (модуль 7) — публиковать состояние по Wi-Fi/MQTT. Для этого уже сейчас API допускает машинную аутентификацию: отдельные «device-токены» (тот же JWT, другой claim kind=device) и эндпоинт `POST /metrics` с валидацией. Расширение — домашнее задание-мост: добавить модель Metric и эндпоинт, а в модулях 6–7 подключаться к нему с реального железа.

```rust
#[derive(Deserialize, Validate)]
pub struct MetricIn {
    #[validate(range(min = -60.0, max = 150.0))]
    pub temperature: f64,
    pub humidity: Option<f32>,
    pub device: String,               // идентификатор узла (pi-1, esp-7...)
}

async fn ingest(State(ctx): State<AppCtx>, dev: DeviceToken,        // device-JWT
                Json(m): Json<MetricIn>) -> Result<StatusCode, AppError> {
    service::metrics::save(&ctx.pool, &dev.device_id, &m).await?;
    Ok(StatusCode::CREATED)
}
```

ЧЕК-ТОЧКА МОДУЛЯ

Лаба 3 сдана и защищена; /metrics готов принимать узлы. Прогоните чек-лист модуля 5 — дальше выходим из комфортного мира Linux.

ДОМАШКА Реализовать /metrics с DeviceToken и SQLite/Postgres-хранилищем; временно протестировать curl'ом фейковыми данными — модуль 6 подменит их реальными.

КОНТРОЛЬ

Чек-лист модуля 5: умею / написал / объясню

- [ ] **Написал** один и тот же мини-API на двух фреймворках и осознанно выбрал стек.
- [ ] **Написал** REST API с auth, DTO, валидацией, миграциями, тестами и compose.
- [ ] **Объясняю** слои hyper/tower/фреймворк и не путаю их с рантаймом Tokio.
- [ ] **Объясняю**, чем query_as! лучше строковых запросов и когда удобен FromRow.
- [ ] **Умею** проектировать DTO и не пропускать внутренности БД наружу.
- [ ] **Умею** защищать эндпоинт типом-экстрактором и обосновать это перед ревьюером.

МОДУЛЬ 6 · НЕДЕЛЯ 13 · ЗАНЯТИЯ 6.1–6.4

Rust на Raspberry Pi

1 неделя | 4 занятия | ≈ 1.5 ч теории | лаба 4: метеостанция

кросс-компиляция · GPIO · I2C

На Raspberry Pi стоит полноценный Linux, поэтому здесь Rust живёт в привычном userspace-режиме: те же крейты, что и на десктопе, плюс доступ к периферии через драйверы ОС. Это модуль-мост: вы применяете всё изученное (владение, Result, async) к физическому миру. Аналоги, на которых всё работает без изменений: Orange Pi, Rock Pi, BeagleBone, любой одноплатник с Linux и GPIO через /dev/gpiod.

НЕДЕЛЯ 13 Кросс-компиляция, периферия, демон

УРОК 6.1 30 мин

Кросс-компиляция: сборка на ноутбуке, запуск на Pi

ЦЕЛЬ — настроить кросс-сборку под ARM и автоматический деплой на плату.

Pi 3/4/5 — 64-битный ARM (target `aarch64-unknown-linux-gnu`), Pi Zero/2W — armv7 или armv6hf. Два пути: `cross` (крейт-обёртка над Docker, всё само) или rustup-target + линкер из пакета gcc-aarch64-linux-gnu (быстрее, но конфиг руками в .cargo/config.toml). Статическая линковка musl-таргетом удобнее для деплоя одним бинарём. Сборка «жирного» Axum-сервиса из лабы 3 кросс-компилируется без правок кода — проверим это прямо на занятии.

ТЕРМИНАЛ · ПУТЬ ЧЕРЕЗ CROSS (ПРОЩЕ)

```
cargo install cross
cross build --release --target aarch64-unknown-linux-gnu
scp target/aarch64-unknown-linux-gnu/release/app pi@raspberrypi.local:~/

<span class="cap">.cargo/config.toml · путь без docker</span>
[target.aarch64-unknown-linux-gnu]
linker = "aarch64-linux-gnu-gcc"

<span class="cap">проверка на плате</span>
pi@raspberrypi:~ $ uname -m      # aarch64
pi@raspberrypi:~ $ ./app         # тот же бинарь, что и на ноутбуке
```

СРАВНЕНИЕ

Go: GOARCH=arm64 go build — проще, потому что рантайм свой. Rust-кросс-компиляция чуть тяжелее из-за C-зависимостей (openssl!), но musl + rustls закрывают 95% случаев. C++: sysroot-танцы с CMake — Rust-кросс проще на порядок.

ДОМАШКА Собрать и запустить на Pi: (а) hello-world статически musl-таргетом, (б) API из лабы 3 с healthz; замерить размер бинарей и время старта; записать в отчёт лабы 4.

УРОК 6.2 30 мин

GPIO: светодиод и кнопка через rppal

ЦЕЛЬ — мигнуть светодиодом и опросить кнопку с обработкой фронта; понять pull-up/pull-down.

Крейт `rppal` — обёртка над gpiod и I2C/UART/SPI Pi: `Gpio::new()` → `get(pin)` → режим Input/Output. Светодиод — просто set_high/set_low; кнопка — Input с подтяжкой (`set_pullup`) и опросом или прерыванием (`set_interrupt` — блокирующее ожидание фронта). Физика: GPIO Pi — 3.3 В, не 5; резистор 220–330 Ом на светодиод обязателен. Владение проявляется буквально: `get(pin)` забирает Pin себе, вернуть в пул — только передав владение обратно `to_owned`-стилем через clear(); типы не дают двум участникам кода держать один пин.

CARGO.TOML

```
[dependencies]
rppal = "0.22"
anyhow = "1"
```

```
<span class="cap">src/bin/blink.rs</span>
use rppal::gpio::Gpio;
use std::thread::sleep;
use std::time::Duration;

fn main() -> anyhow::Result<()> {
    let gpio = Gpio::new()?;
    let mut led = gpio.get(17)?.into_output();     // BCM-нумерация!
    loop {
        led.set_high(); sleep(Duration::from_millis(300));
        led.set_low();  sleep(Duration::from_millis(300));
    }
}
```

```
<span class="cap">src/bin/button.rs — прерывание по фронту</span>
use rppal::gpio::{Edge, Gpio};

fn main() -> anyhow::Result<()> {
    let gpio = Gpio::new()?;
    let mut btn = gpio.get(27)?.into_input_pullup();
    loop {
        btn.wait_for_edge(Edge::Falling)?;         // блокирует до нажатия
        println!("нажато!");
        std::thread::sleep(Duration::from_millis(200));  // антидребезг
    }
}
```

> СРАВНЕНИЕ
>
> C++: WiringPi/bcm2835 — сырые регистры, ошибки адреса = UB. Go: periph.io. Rust rppal: типы различают Input/Output-пин на уровне компиляции — переключить режим нельзя по ошибке, нужен новый get().

ПРАКТИКА Собрать схему «кнопка управляет светодиодом»: interrupt-поток пишет в mpsc, главный цикл тумблит пин; отсюда начинается привычка «периферия в потоке, логика в данных».

ДОМАШКА Бегущий огонь на 4 светодиодах с регулируемой скоростью (кнопки + и −); тайминг через std::time, никаких busy-loop.

УРОК 6.3 30 мин

I2C/UART: читаем датчик BMP280

ЦЕЛЬ — подключить датчик по I2C и прочитать температуру/давление; понять адресацию устройств.

Шина I2C: два провода (SDA/SCL), каждый slave имеет 7-битный адрес — BMP280 обычно 0x76/0x77. Скан шины: `i2cdetect -y 1`. Драйвер датчика — крейт `bme280-rs` (embedded-hal-совместимый!) поверх rppal-реализации I2C; заметьте: этот же драйвер работает и на ESP32 (модуль 7) — потому что оба хала реализуют один трейт embedded-hal. Это главный архитектурный трюк embedded-Rust, и мы его здесь впервые трогаем. UART — `rppal::uart` для связи с внешними модулями (GPS, GSM) — оставляем как опцию проекта.

CARGO.TOML

```
[dependencies]
rppal = "0.22"
bme280 = "0.5"
linux-embedded-hal = "0.4"

<span class="cap">src/bin/sensor.rs</span>
use bme280::i2c::BME280;
use linux_embedded_hal::{I2cdev, Delay};
use rppal::i2c::I2c;

fn main() -> anyhow::Result<()> {
    let i2c_dev = I2c::new()?;                    // /dev/i2c-1
    let mut sensor = BME280::new_primary(i2c_dev);
    sensor.init(&mut Delay {})?;
    loop {
        let m = sensor.measure(&mut Delay {})?;
        println!("{:.1} °C, {:.1} гПа, {:.1} %",
            m.temperature.celsius(), m.pressure.hpa(), m.humidity.percent());
        std::thread::sleep(std::time::Duration::from_secs(5));
    }
}
```

КЛЮЧЕВАЯ МЫСЛЬ

`bme280` не знает, Linux это или микроконтроллер: он требует «что-то, реализующее embedded-hal::i2c::I2c». На Pi это rppal/linux-embedded-hal, на ESP32 — esp-hal. Один драйвер — два мира.

ДОМАШКА Датчик по I2C: вывести 20 измерений с таймстампами в CSV; проверить `i2cdetect` до подключения и после; документ «схема подключения» с фотографией макетки — обязательная часть лабы 4.

УРОК 6.4 30 мин

Демон: systemd, логи и отправка в API лабы 3

ЦЕЛЬ — превратить скрипт в сервис: автозапуск, перезапуск, journald-логи, отправка метрик в свой API.

Прод-демон на Pi — это systemd-unit: Restart=always, лимиты памяти, журнал через journald. Наш демон читает датчик (поток-читатель), агрегирует минуты (среднее/мин/макс через алгоритмические навыки модуля 3) и POST-ит в /metrics лабы 3 с device-токеном; при

отсутствии сети — буфер в файл и дозагрузка. Это первый сквозной кусок семестрового проекта: физика → агрегация → сеть → БД.

/ETC/SYSTEMD/SYSTEM/METEO.SERVICE

```
[Unit]
Description=Rust meteo agent
After=network-online.target

[Service]
ExecStart=/home/pi/meteo-agent
Environment=RUST_LOG=info
Environment=API=http://192.168.1.10:8080
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
<span class="cap">терминал</span>
sudo systemctl daemon-reload && sudo systemctl enable --now meteo
journalctl -u meteo -f          # живые логи
```

ЛАБОРАТОРНАЯ РАБОТА №4 · 8% ИТОГОВОЙ ОЦЕНКИ

Метеостанция на Pi: датчик → агрегация → REST

- **Минимум:** кросс-сборка; чтение датчика по I2C; поминутная агрегация; POST /metrics в API лабы 3; systemd-юнит; фото макетки и README.
- **Продвинутый:** локальный буфер при потере сети (CSV + дозагрузка), кнопка «принудительно отправить», health-светодиод.
- **Со звёздочкой:** второй датчик по UART или вторая Pi — «сеть» из двух агентов.

ДОМАШКА Сдать лабу 4: демон живёт после reboot, метрики видны в API, журнальный фрагмент journald приложен. Прогнать чек-лист модуля 6.

КОНТРОЛЬ

Чек-лист модуля 6: умею / написал / объясню

- ☐ **Настроил** кросс-компиляцию aarch64 и деплой одним бинарём.
- ☐ **Написал** blink + кнопка (interrupt) + чтение датчика по I2C.
- ☐ **Написал** systemd-демон с отправкой метрик в свой API.
- ☐ **Объясняю** embedded-hal: один драйвер датчика на Linux и на MK.
- ☐ **Объясняю** pull-up/pull-down, фронты, антидребезг.

МОДУЛЬ 7 · НЕДЕЛЯ 14 · ЗАНЯТИЯ 7.1–7.4

ESP32 и no_std: bare-metal на Rust

1 неделя | 4 занятия | ≈ 1.5 ч теории | лаба 5: IoT-узел | esp-hal · Embassy · Wi-Fi

ESP32 — микроконтроллер без ОС: ваш код и есть прошивка. Мир no_std — самый честный мир Rust: нет аллокатора по умолчанию, нет std, каждый килобайт на счету, и именно поэтому здесь Rust принято чаще, чем C. Стек esp-rs: форк компилятора (ставится через `espup`), крейт `esp-hal` с Embassy-async, `esp-wifi` для беспроводного стека. Аналоги-платы: ESP32-S3, ESP32-C3/C6, а также RP2040/Raspberry Pi Pico W — курс по ним переносится практически один-в-один.

НЕДЕЛЯ 14 no_std, Embassy, Wi-Fi

УРОК 7.1 30 мин

Bare-metal мышление: что меняется без std

ЦЕЛЬ — понять разницу std/no_std, роль alloc и структуру esp-rs проекта.

В no_std нет std::fs/net/thread — есть core и (по желанию) alloc со Vec/String через глобальный аллокатор. Отсутствие ОС означает: точка входа — Reset-вектор, до вашего main работает startup-код HAL; panic! в прошивке — это ловушка-обработчик (defmt-panic-halt: замираем с логом). Асинхронность Embassy заменяет ОС: executor живёт в вашей прошивке, задачи — Future-автоматы из урока 4.3, но теперь их может быть сотни при памятью в десятки килобайт. Инструментарий: `espup install` (форк-тулчейн), `espflash` для прошивки и мониторинга.

ТЕРМИНАЛ · РАЗОВАЯ НАСТРОЙКА

```
cargo install espup espflash
espup install                       # тулчейн Xtensa/LLVM
espflash board-info                 # плата видна? OK

<span class="cap">структура проекта (esp32c3)</span>
# cargo generate esp-rs/esp-template   # оффициальный шаблон
.cargo/config.toml    # target = riscv32imc-esp-espidf? нет: riscv32imc-unknown-none-elf
src/main.rs           # #[entry], hal-инициализация
.cargo/               # память, флаги чипа
```

СРАВНЕНИЕ

C++-embedded: Arduino-стиль с setup/loop и скрытой malloc. Rust embedded: явный hal-периферийный доступ через singletons (ownership периферии — типы!), Embassy вместо RTOS, defmt вместо printf. Go здесь не живёт вообще — рантайм не влезает.

ДОМАШКА Собрать и прошить шаблон esp-template для вашей платы; в комментарии к коммиту — модель чипа, доступная память RAM/Flash из вывода espflash.

УРОК 7.2 30 мин

Первая прошивка: blink на esp-hal

ЦЕЛЬ — написать blink с осознанным пониманием каждого шага инициализации.

Классический blink на esp-hal — это периферийный синглтон: `init()` конфигурирует CPU/такты и забирает периферию во владение; GPIO-пины вынимаются из `io`-объекта по одному — взять один пин дважды невозможно, компилятор гарантирует. Логи — `defmt`: строка формата передаётся на хост компактно, прошивка не тянет printf-стек. Прошивка: `espflash flash target/riscv32imc-unknown-none-elf/release/blink --monitor`.

SRC/MAIN.RS · ESP-HAL 0.23+, ЧИП ESP32-C3

```rust
#![no_std]
#![no_main]

use esp_hal::{
    gpio::{Io, Level, Output},
    prelude::*,
    delay::Delay,
};
use esp_println::println;

#[entry]
fn main() -> ! {
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let io = Io::new(peripherals.GPIO, peripherals.IO_MUX);

    let mut led = Output::new(io.pins.gpio2, Level::Low);
    let mut delay = Delay::new();

    loop {
        led.set_high();
        println!("on");
        delay.delay_millis(300);
        led.set_low();
        println!("off");
        delay.delay_millis(300);
    }
}

#[panic_handler]
fn panic(_: &core::panic::PanicInfo) -> ! {
    esp_println::println!("PANIC");
    loop {}                                  // halt: ресет кнопкой/по питанию
}
```

> ПРО ВЕРСИИ
>
> API esp-hal активно развивается; точные имена функций сверяйте с версией в Cargo.toml шаблона esp-template. Принципы (синглтоны-владение, no_std, entry/panic) не меняются — меняется синтаксис инициализации. В лабе фиксируйте версии зависимостей lock-файлом.

ДОМАШКА Секундный софт-таймер на 4 светодиодах (двоичный счётчик на пинах) + вывод номера счётчика в лог; замерить размер прошивки (`cargo size`).

УРОК 7.3 30 мин

Embassy: async-задачи и датчик по I2C

ЦЕЛЬ — запустить два Embassy-таска (blink + датчик) и увидеть экономию против потоков ОС.

Embassy — async-исполнитель для МК: задачи объявляются макросом `#[embassy_executor::task]`, спавнятся в main-инициализаторе, таймеры — `Timer::after`. Датчик — тот же крейт bme280 из модуля 6, только теперь с esp-hal-реализацией I2C: трюк embedded-hal в полную силу. Замер: две задачи в no_std на C3 занимают меньше памяти, чем один поток FreeRTOS; 10 задач Embassy — норма, 10 потоков ОС — нет. Именно так строится реальный IoT-узел: мигалка-«сердцебиение», читатель датчика, сетевой агент.

ЭСКИЗ ДВУХ ЗАДАЧ (ESP-HAL + EMBASSY, ВЕРСИИ ПО ШАБЛОНУ)

```
#[embassy_executor::task]
async fn heartbeat(mut led: Output<'static>) {
    loop {
        led.toggle();
        Timer::after_millis(500).await;
    }
}

#[embassy_executor::task]
async fn sensor_loop(i2c: I2C<'static, I2C0>) {
    let mut bme = BME280::new_primary(i2c);
    // init + цикл измерений с Timer::after_secs(5).await
}

#[main]
async fn main(spawner: Spawner) -> ! {
    // hal-init, io, i2c, led
    spawner.spawn(heartbeat(led)).ok();
    spawner.spawn(sensor_loop(i2c)).ok();
    loop { Timer::after_secs(3600).await; }
}
```

> СРАВНЕНИЕ
>
> Go-разработчику: это «горутины», но в 40 КБ RAM. C++-embedded: RTOS-потоки со стеками в килобайтах и приватные мьютексы; Embassy-задачи делят один стек и не требуют mutex на общих данных — заёмные правила действуют и здесь.

ДОМАШКА Две задачи + лог defmt: heartbeat-счётчик и измерения датчика; проверить, что заёмы между задачами невозможны без Arc/Mutex — как организовали обмен? (Ответ должен быть: сообщение через канал Embassy.)

УРОК 7.4 30 мин

Wi-Fi и сеть: esp-wifi, HTTP/MQTT-публикация

ЦЕЛЬ — подключить узел к сети и отправить измерение в свой API: замкнуть семестровый контур.

`esp-wifi` добавляет Wi-Fi-стек (STA-режим: ssid/пароль, DHCP) и может поднять Embassy-совместимые сокеты. Публикация — HTTP POST на /metrics лабы 3 с device-токеном (реализация на крейте `reqwless` для no_std) или MQTT-брокеру (крейт `rust-mqtt`) с мостом-агентом на Pi/сервере, который кладёт данные в API. Требование энергобюджета: сон между

публикациями (`esp_sleep`/deep-sleep) — обсуждаем концептуально, реализуем «со звёздочкой».

ЭСКИЗ СЕТЕВОГО ТАСКА (УПРОЩЁННО; API ПО ВЕРСИИ ESP-WIFI)

```
async fn net_loop(controller: WifiController<'static>, cfg: NetCfg) {
    // 1) wifi init + set_configuration(ssid, psk) + connect; лог статуса
    // 2) раз в 60 с: измерение → serde_json → reqwless POST cfg.api/metrics
    //    с Authorization: Bearer <device-jwt>
    // 3) при ошибке сети: ретрай с экспоненциальной паузой
    loop {
        Timer::after_secs(60).await;
        // ...
    }
}
```

ЛАБОРАТОРНАЯ РАБОТА №5 · 8% ИТОГОВОЙ ОЦЕНКИ

IoT-узел на ESP32: датчик → Wi-Fi → API

- **Минимум:** no_std-прошивка: blink, датчик по I2C, Wi-Fi-подключение, POST /metrics в API лабы 3 с device-токеном; логи defmt; README со схемой и версией HAL.
- **Продвинутый:** буфер измерений во flash/RTC-память при потере сети, кнопка-вкл/выкл публикации, экспоненциальный ретрай.
- **Со звёздочкой:** deep-sleep между публикациями с замером тока (USB-док), либо MQTT + агент-мост.

ДОМАШКА Сдать лабу 5: живые данные узла видны в API; приложить лог подключения и таблицу версий крейтов. Прогнать чек-лист модуля 7.

КОНТРОЛЬ

Чек-лист модуля 7: умею / написал / объясню

- [ ] **Собрал и прошил** no_std-проект на ESP32; объясняю каждый шаг инициализации.
- [ ] **Написал** две Embassy-задачи с обменом через канал.
- [ ] **Написал** Wi-Fi-узел, публикующий измерения в свой API.
- [ ] **Объясняю**, почему один драйвер bme280 работает и на Pi, и на ESP32.
- [ ] **Объясняю** panic-handler и что происходит при панике на МК.

МОДУЛЬ 8 · НЕДЕЛИ 15–16 · ДВА СПРИНТА

Семестровый проект

2 спринта | 35% оценки | сквозной IoT-сценарий | защита: демо 10 мин + ревью

Проект собирает весь курс в одну систему: узлы на ESP32 и/или Raspberry Pi отправляют данные в API на Axum, данные хранятся в PostgreSQL, алгоритмическая библиотека обрабатывает их (агрегация, детекция аномалий, фильтрация), всё покрыто тестами и разворачивается одной командой. Проект не обязан быть «уникальным исследованием» — он

обязан быть работающим и объяснимым: каждый компонент вы можете защитить на вопрос «почему так».

СПРИНТ 1 · НЕДЕЛЯ 15 Архитектура и MVP

Первые два дня — выбор трека и план. Три стандартных трека; допустимы собственные при согласовании с наставником не позднее первого дня спринта. Дальше — ежедневные мини-коммиты и чекпоинт в конце недели: работающий MVP и «карта системы» на один слайд.

Трек	Состав системы	Ключевая сложность
A · Сервис	API лабы 3 + алгоритмический модуль (аномалии, агрегации) + клиенты-симуляторы узлов на Tokio	Архитектура сервиса, конкурентная запись, тесты
B · Pi-сеть	2–3 агента на Pi/Raspberry Pi Zero с разными датчиками → API → БД → дашборд (простая HTML-страница или JSON-отчёт)	Деплой на несколько плат, буферизация, systemd-надёжность
C · IoT полный	ESP32-узел + Pi-агент → API → БД + алгоритмическая обработка (пороговые алерты, скользящее окно)	Сквозная интеграция, no_std-ограничения, энергобюджет

ЧЕКПОИНТ СПРИНТА 1

В репозитории проекта: README с архитектурной картинкой (можно схемой из блоков), MVP собирается одной командой, узел шлёт хотя бы один тип данных, API принимает и хранит. Демо-заметка в issues: что работает, что сломано, план на спринт 2.

СПРИНТ 2 · НЕДЕЛЯ 16 Полировка, документация, защита

Спринт 2 — превращение MVP в сдачу: тесты, обработка краёв (обрыв сети, рестарт БД, пустые данные), README с инструкцией запуска «с нуля за 10 минут», демо-сценарий на 10 минут. Защита: демо на живом железе (или видео при необходимости) + код-ревью с вопросами по коду: ownership-решения, выбор трейтов, границы async, причины выбора структур данных. Оценка — по таблице; отдельно оценивается честность ограничений: страница «известные ограничения» ценится выше, чем её отсутствие.

Критерий защиты	Баллы	Что смотрим
Работоспособность демо	10	Всё поднимается по README; сквозной сценарий проходит на живой системе
Архитектура и идиоматичность	10	Слои, типизация состояний, отсутствие clone-мусора, объяснимые трейды
Тесты и надёжность	5	Unit + интеграционные; поведение при обрывах; graceful shutdown
Документация и защита	10	README, схема, демо-речь, ответы «почему», страница ограничений

- [ ] **Сдача проекта** — ссылка на репозиторий + 10-минутное демо + ответы на ревью.
- [ ] **Автомат-условие** — ≥ 90 баллов итога и сданные все 5 лаб без долгов: защита превращается в публичное демо без оценки.

ПОСЛЕ КУРСА

Куда двигаться дальше

Четыре продолжения, отсортированные по цене входа: (1) **WebAssembly** — Rust в браузере и на edge (wasm-bindgen, Leptos/Yew) — ваш веб-модуль даёт половину базы; (2) **низкоуровневые системы** — написание аллокатора, файловой системы, загрузчика; книга Embedded Rust Discovery и rustc internals; (3) **продакшен-инфраструктура** — метрики (prometheus), otel-трейсинг, профилирование (cargo flamegraph), fuzzing (cargo-fuzz) — прямое продолжение модуля 5; (4) **безопасность** — аудит unsafe-кода, miri, склонение к структуре «небольшой безопасный core + audited unsafe». Портфолио из курса уже содержит 5 лаб и проект — оформите их как публичные репозитории: это рабочие артефакты собеседований.

СПРАВОЧНИК

Глоссарий

`alloc`
Крейт с типами, требующими аллокатора (Vec, String, Box); доступен в no_std-прошивках при подключённом #[global_allocator].

`async fn / Future`
Функция, возвращающая конечный автомат-будущее; исполняется только при опросе (poll) рантаймом (Tokio, Embassy).

`borrow checker`
Часть компилятора, проверяющая правила ссылок: N читателей XOR 1 писатель; ссылка живёт не дольше владельца.

`Box<T>`
Владелец значения в куче; разрывает рекурсию типов, даёт конечный размер.

`Cargo.lock`
Фиксация точных версий зависимостей; коммитится для бинарей, не коммитится для библиотек.

`clone()`
Явная глубокая копия; в идиоматичном коде встречается осознанно и с комментарием, а не как «лечебное средство».

`Coercion (deref)`
Авто-приведение &String → &str, &Vec<T> → &[T] при передаче аргументов.

`crates.io`
Центральный реестр пакетов; «blessed» список рекомендуемых крейтов — blessed.rs.

`defmt`
Компактный фреймворк логирования для embedded: формат-строки не хранятся в прошивке целиком.

DST
Dynamically Sized Type — тип неизвестного размера (&str, dyn Trait); живёт за указателем.

dyn Trait
Трейт-объект: динамическая диспетчеризация через vtable; требует object safety.

embedded-hal
Трейты периферии (I2C, SPI, GPIO); драйверы пишутся один раз под любой чип, реализующий эти трейты.

Elision
Правила автоматического вывода lifetime в сигнатурах, когда аннотацию можно не писать.

Embassy
Async-исполнитель для микроконтроллеров; задачи — Future-автоматы без стеков ОС.

esp-hal / esp-wifi
HAL и Wi-Fi-стек для ESP32-линейки от проекта esp-rs.

entry API
Атомарный «найди или создай» для HashMap: entry(k).or_insert(v).

From / Into
Трейты конверсии; реализуешь From — Into получаешь бесплатно; поддерживает «?».

HAL
Hardware Abstraction Layer — крейт-обёртка над регистрами чипа.

IntoResponse
Трейт Axum: любой ваш тип ошибки/ответа становится HTTP-ответом.

lifetime 'a
Параметр, описывающий связь времени жизни ссылок; не продлевает ничью жизнь.

Move semantics
Передача владения по умолчанию; источник — уникальность владельца.

NLL
Non-Lexical Lifetimes: заём живёт до последнего использования, а не до конца блока.

no_std
Режим сборки без std: только core (+alloc при наличии аллокатора); среда МК.

Option<T>
Явное «значение может отсутствовать»; замена null.

PAC
Peripheral Access Crate — сгенерированный из SVD доступ к регистрам чипа; основа HAL.

panic!
Невосстановимый сбой потока; в МК — halt/ресет; ожидаемые сбои — через Result.

Result<T, E>
Явный исход операции; «?» — синтаксис раннего возврата ошибки.

Rc / Arc
Счётчик ссылок (однопоточный / атомарный); циклы — утечки, лечатся Weak.

RefCell
Внутренняя мутабельность: правила заёмов проверяются в рантайме (паника вместо UB).

Send / Sync
Маркерные трейты потокобезопасности: можно передать в поток / можно шарить по &.

serde
Дефакто-стандарт сериализации: derive Serialize/Deserialize, форматы JSON/TOML/...

SQLx
Async-драйвер БД с compile-time проверкой SQL-запросов.

Tokio
Мультипоточный async-рантайм: задачи, таймеры, каналы, io.

Trait
Контракт поведения; статическая (генерики) и динамическая (dyn) диспетчеризация.

unsafe
Блок, где вы берёте на себя инварианты Rust; в прикладном коде — редкость и комментарий-обоснование.

Waker / poll
Механизм Future: рантайм опрашивает автомат, задача будит рантайм через Waker.

zero-cost abstraction
Абстракции (генерики, итераторы), компилирующиеся в тот же код, что и ручной.

СПРАВОЧНИК

Ресурсы курса

КАНОНИЧЕСКАЯ ТЕОРИЯ	The Rust Programming Language («The Book») — doc.rust-lang.org/book; русское издание сообщества — rust-lang-ru.github.io. Основное чтение недель 1–4.
ТРЕНАЖЁР	Rustlings (github.com/rust-lang/rustlings) — 100+ микроупражнений под каждую тему; идеальная домашка «на разминку».
ПРИМЕРЫ КОДА	Rust by Example — doc.rust-lang.org/rust-by-example; уместно как шпаргалка по синтаксису.
EMBEDDED	The Embedded Rust Book и Discovery — docs.rust-embedded.org; esp-rs book — docs.esp-rs.org; Embassy book — embassy.dev.
ASYNC	Tokio tutorial — tokio.rs/tokio/tutorial; страница async book (rust-lang.github.io/async-book) — про Future изнутри.
API-ДОКИ	docs.rs — документация всех крейтов с исходниками; axum/SQLx примеры — официальные examples в репозиториях.
АЛГОРИТМЫ	Ваши собственные algolib +Criterion-отчёты; для тренировки — exercism.org/tracks/rust и codewars kata на Rust.
ИНСТРУМЕНТЫ	rustup, cargo, clippy, rustfmt, cargo-testify-набор: cargo doc, cargo tree, cargo bloat, cargo flamegraph, miri.
ВЫБОР КРЕЙТОВ	blessed.rs — сообщество-поддерживаемый список проверенных крейтов по доменам; lib.rs — каталог с метриками.

компилятор — не враг. он — самый строгий ревьюер, который у вас будет.

За 16 недель вы прошли путь от «почему не компилируется» до прошивки на микроконтроллере, который шлёт данные в ваш же REST-сервис. Дальше — только практика: рабочие задачи, side-проекты, чтение чужого кода. cargo new — и вперёд.